

MCP 기반 5G 슬라이스 자율 오케스트레이션 설계서

C · 에이전트

판단 로직 · 도구 호출 루프 · 목 서버

담당	오케스트레이터 · 프롬프트 · 계약 검사
구성	공통 계약 + 담당 상세 (—페이지)
저장소	DBAJK/SRM-main-mcp
작성	2026-09-21

이 본은 전체 설계서에서 **C 담당분**을 뽑은 것이다. 앞의 **공통 계약**(공통 타입 · 도구 일람 · 데이터 연쇄 · 제어 루프 · 오류 규약 · 금지 필드)은 **3명이 동일하게 읽는다**. 그 부분을 건너뛰면 자기 서버만 보고 계약을 깬다.

내 범위

전 서버	21개 도구 전부를 호출한다 — 본문에 명세 전체를 수록
직접 작성	srm_mcp/mock/ 5개 · orchestrator/ · check_contract.py

내가 생산하는 값 — 남이 여기에 의존한다

값	출처	소비자	비고
run_id	C 발급	A(①) reset · A(④) 전체	없으면 60회 실행이 섞인다
situation	C 추론	B(②) · A(④)	관측에서 추론. 어떤 도구도 주지 않음
confidence.situation	C 판단	A(④)	상황 오판 가능성 — 유일한 경로
confidence.combined	C 계산	A(④)	세 값의 기하평균
정책 선택	C 판단	B(②) policy 인자	신뢰도 비교 결과
에스컬레이션 판단	C 판단	A(④)	combined < $\tau(0.45)$

내가 소비하는 값 — 없으면 내가 막힌다

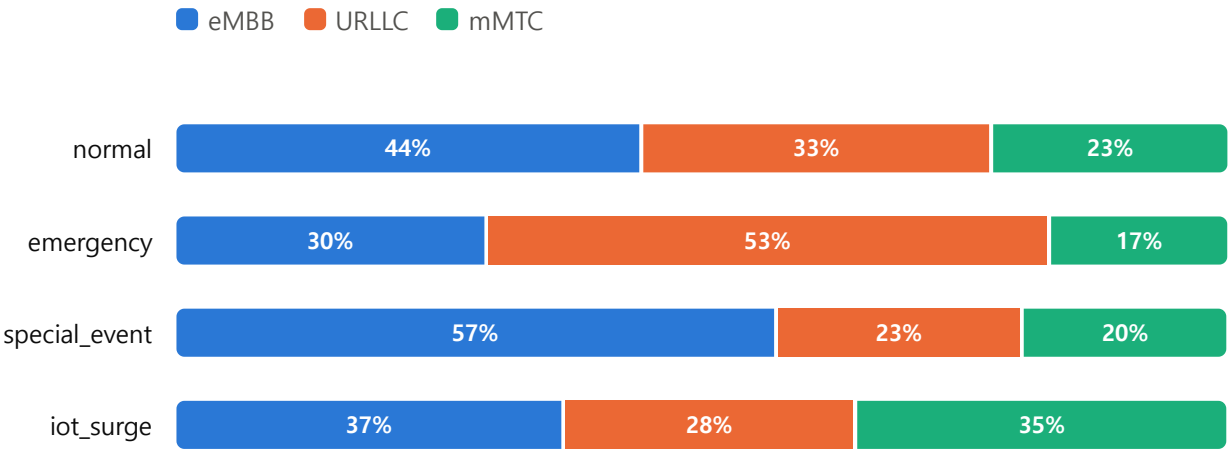
값	생산자	쓰는 곳
Observation · ApplyResult	A(①)	판단 · 실행
PolicyProposal · ScoredOffering	B(②③)	정책·벤더 선택
Outcome · recent_error	B(⑤)	다음 판단에 반영
decision_id · fallback	A(④)	채점 · 에스컬레이션 후 진행

서버끼리는 직접 대화하지 않는다. 위의 모든 값은 오케스트레이터(C)가 옮겨 심는다. 내 도구가 남의 도구를 부르는 코드는 어디에도 없어야 한다 — 하나라도 있으면 §2.2 규약 위반이고, 무상태 서버가 상태를 갖게 된다.

C가 알아야 할 환경 특성

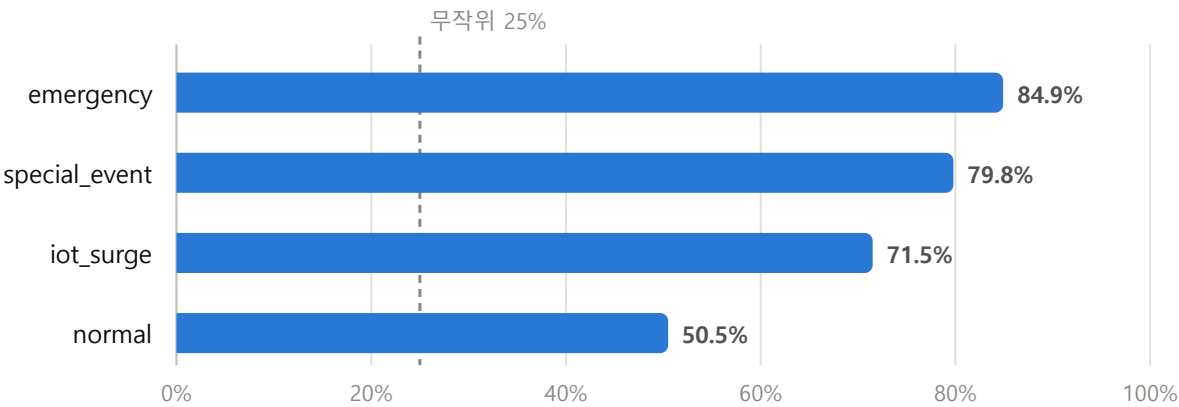
프롬프트 설계의 전제다. 에이전트가 무엇을 근거로 판단할 수 있는지가 여기서 정해진다.

트래픽 구성비 — 상황 인지의 실제 근거



시간 요인(±30%)이 이벤트 효과와 비슷한 크기라 **절대량으로는** 평소 피크와 비상이 구분되지 않는다. 세 슬라이스에 같은 배수가 곱해지므로 구성비는 시간 요인에 불변이며, 여기서 신호가 나온다. **프롬프트가 절대량을 보게 유도하면 안 된다.**

구성비만으로 얻는 상황 인지 정확도



전체 **71.7%**가 단일 스텝 참조점이다. 에이전트는 시계열 `violations` · `classify_demand` 를 함께 보므로 더 잘할 수 있다. `normal` 50.5%는 **오탐이 많다**는 뜻이며, `confidence.situation` 을 에스컬레이션 경로에 넣어야 하는 근거다.

공통 타입

공통 계약 · `claude/spec/common.md`

모든 서버가 공유한다. `mcp/common/schema.py` 에 pydantic 모델로 한 번만 정의한다.

SliceTriple

3개 슬라이스에 대한 실수 3개. **배열이 아니라 딕셔너리다.** (배열은 인덱스 순서가 바뀌면 조용히 틀린다)

```
{"embb": 0.40, "urllc": 0.40, "mmtc": 0.20}
```

SliceFlags

```
{"embb": true, "urllc": true, "mmtc": false}
```

열거형

이름	값	비고
Situation	"normal" · "emergency" · "special_event" · "iot_surge"	에이전트가 관측으로부터 추론해 만드는 값. 어떤 도구도 알려주지 않는다
PolicyName	"rule_based" · "lstm_forecast" · "dqn"	
SliceType	"eMBB" · "URLLC" · "mMTC"	③에서만 쓴다. 대소문자가 <code>vendors.json</code> 의 키와 정확히 일치해야 한다
Status	"ok" · "unavailable" · "error"	

숫자 타입 규약 ⚠

JSON은 numpy를 모른다. 모든 경계에서 파이썬 기본형으로 변환한다.

타입	<code>json.dumps()</code>	비고
<code>np.ndarray</code>	실패	<code>.tolist()</code> 필요
<code>np.bool_</code>	실패	<code>bool()</code> 필요
<code>np.float64</code>	통과	파이썬 <code>float</code> 상속. 우연히 통과하는 것
<code>float</code> , <code>bool</code> , <code>int</code>	통과	

`violations` 가 정확히 `np.bool_` 이다 (`ml_orchestrator_demo.py:563`). 그대로 반환하면 도구 호출이 예외로 죽는다. `np.float64` 는 통과하므로 한참 잘 돌다가 `violations` 에서만 터진다.

도구 일람

공통 계약 · `claude/spec/tools.md`

5개 서버 · 도구 21개. 상세 계약은 `spec/<server>.md`.

서버	도구	입력	출력	부작용
① observe	get_observation	—	Observation	없음
	step	n	Observation	상태 전진
	apply_allocation	SliceTriple	ApplyResult	배분 변경
	get_history	n	HistoryBlock	없음
	add_capacity	slice_type, amount, expires_at_step, slice_id	CapacityState	용량 증설
	reset	run_id, scenario, seed	Observation	에피소드 초기화
② policy	list_policies	—	[PolicyInfo]	없음
	propose_allocation	policy, Observation, situation, history, recent_error	PolicyProposal	없음
	compare_policies	Observation, situation, history, recent_errors	[PolicyProposal]	없음
	classify_demand	Observation	DemandClass	없음
	list_offerings	slice_type, region	[Offering]	없음
③ market	score_offerings	slice_type, qos	[ScoredOffering]	없음
	explain_score	vendor_id, slice_type, qos	ScoreBreakdown	없음
	procure	vendor_id, slice_type, qos, duration_steps, current_step	Procurement	조달 기록
	update_rating	vendor_id, outcome	RatingUpdate	vendors.json 갱신
	record_decision	판단 일체	{decision_id}	기록
④ audit	record_escalation	판단 + 사유	{escalation_id, decision_id, fallback}	기록 = 개입 1회 + 폴백 결정
	get_decisions	n, kind	[Decision]	없음
	get_metrics	window	Metrics	없음

서버	도구	입력	출력	부작용
⑤ feedback	report_outcome	decision_id , Observation	Outcome	신뢰도 갱신
	get_reliability_table	—	ReliabilityTable	없음

부작용 있는 도구 8개는 한 스텝에 정해진 횟수만 호출된다.

step · apply_allocation · record_decision · report_outcome 은 각 1회,
record_escalation · procure · add_capacity 는 0 또는 1회.

1스텝 표준 호출 순서

```

1. ①.get_observation()                → obs
1b. ①.get_history(10)                 → history      ← 빠뜨리면 LSTM이 영영 unavailable
2. ②.classify_demand(obs)              → dominant      (선택)
   ⑤.get_reliability_table()           → effective, recent_error
   [LLM] situation · confidence.situation 추론
3. ②.propose_allocation(policy, obs, situation, history, recent_error)
   [파이썬] combined = (situation × intrinsic × effective)^(1/3)

   ┌ combined < 0.45 → ④.record_escalation(...)
   │                  → escalation_id, decision_id, fallback_allocation
   │                  **4.5를 건너뛰고 6으로.** allocation ← fallback_allocation
   │                  (record_decision을 또 부르면 같은 스텝에 decision 레코드가 둘)
   └ 계속 ↓

4. [LLM 조달 판단 · demand_pressure ≥ 1.0일 때만]
   ③.score_offerings → ③.procure(current_step=obs.step) → ①.add_capacity
5. ④.record_decision(..., slice_id, vendor_id) → decision_id      (apply 전에!)

6. ①.apply_allocation(**allocation)      → normalized, delta
7. ①.step()                             → obs+1
8. ⑤.report_outcome(decision_id, obs+1)   → sla_met, vendor_id
9. [vendor_id 있으면] ⑤.update_rating(vendor_id, {sla_met, decision_id})

```

조달이 기록보다 앞서는 이유 — record_decision 이 slice_id · vendor_id 를 인자로 받기 때문이다
(설계서 §3.1 규칙 4). 뒤집으면 결정 레코드에 vendor_id 가 없고, ⑤ report_outcome 이 그 레코드에서
vendor_id 를 찾으므로 항상 null 이 되어 ⑤→③ 레이팅 되먹임이 통째로 죽는다. "판단했으나 실행하지 못한
경우를 남긴다"는 규칙 3의 목적은 훼손되지 않는다 — 조달 실패도 slice_id: null + rationale 로 기록되기
때문이다.

add_capacity 는 반드시 step() (7) 전에 부른다. 뒤로 가면 용량이 다음 스텝부터 반영되어 조달한 바로 그
스텝은 효과를 못 본다 (W1).

1b를 빠뜨리면 lstm_forecast 가 영영 status: "unavailable" (history_insufficient)이 되어 정책 선택
축이 죽는다. history 는 3단계의 인자인데 얻는 호출이 순서에 없으면 null 로 넘어간다.

에스컬레이션 분기는 4·5를 건너뛰는다. `record_escalation` 이 이미 `decision_id` 와 `fallback_allocation` 을
내므로 `record_decision` 을 또 부르면 같은 스텝에 `kind: "decision"` 레코드가 둘 생기고
`sum(policy_usage)` 불변식이 깨진다 (`spec/audit.md`).

루프는 파이썬이 돌린다. LLM은 2·4단계에서만 호출하며 답하는 값은 넷뿐이다 — `situation` ·
`confidence.situation` · 정책 선택 · 조달 여부. `combined` 와 에스컬레이션 여부는 파이썬이 계산한다. 근거는
`flow/loop.md` .

서버끼리 직접 대화하지 않는다. 모든 값은 에이전트를 통과한다. 전체 도식과 출력→입력 대응표는 `flow/data-chain.md` .

제어 루프 계약

공통 계약 · `claude/flow/loop.md`

1스텝의 시간 규약 · 액추에이터 · 고정 루프 · 컨텍스트 · 에스컬레이션 정의

3. 제어 루프 계약

분리 설계서에 없던 부분이고, 측정 지표의 정의가 전부 여기에 달려 있다.

3.1 시간 규약

원 코드의 `run_step()` (:535)은 한 스텝 안에서 트래픽 생성 → 이용률 계산 → 배분 갱신을 모두 한다. MCP에서는 배분 갱신이 에이전트 쪽으로 나가므로 이 순서를 쪼개야 하고, 쪼개는 지점이 곧 지표의 정의가 된다.

```
# ml_orchestrator_demo.py:545
utilization = self.update_utilization(traffic)    # ← 이 시점의 allocation은 직전 스텝의 것
...
allocation = self.update_allocation*(...)         # ← 그 다음에 갱신
```

즉 이용률은 언제나 "직전 결정의 결과" 다. 이 성질을 그대로 살린다.

```
reset(run_id, scenario, seed)
└─ alloc ← [0.4, 0.4, 0.2], cap ← [1, 1, 1], traffic 생성
   util = traffic / (alloc × cap)

— 스텝 t —
1. obs_t = ①.get_observation()           # util_t 는 a_{t-1}의 성적표
2. [에이전트 판단] ②.classify_demand / propose_allocation
   ③.get_reliability_table
2b. [조달 분기] obs_t.demand_pressure ≥ 1.0 일 때만
   ③.score_offerings → ③.procure(..., current_step=t)
   ③.add_capacity(amount=capacity_gain, expires_at_step)
3. ④.record_decision(..., slice_id, vendor_id) → decision_id
4. ①.apply_allocation(a_t)               # 액추에이터. 평활·클립·정규화
5. ①.step()                             # 만료 회수 → traffic_{t+1}
   # util_{t+1} = traffic_{t+1} / (a_t × cap_{t+1})
6. ⑤.report_outcome(decision_id, obs_{t+1})
```

따르는 규칙 네 가지

- 결정의 채점은 1스텝 지연된다. t의 결정은 t+1의 관측으로 평가된다. `report_outcome` 은 반드시 `step()` 이후에 호출한다.
- 에피소드 마지막 결정은 채점되지 않는다. 지표 집계에서 제외한다. 60스텝이면 유효 표본 59.
- 기록은 배분 실행(4)보다 먼저다. 에이전트가 판단만 하고 실행에 실패한 경우도 ④에 남는다. 실행 후 기록이면 실패 사례가 사라져 자율 처리율이 과대평가된다.

4. 조달(2b)만 기록보다 앞선다. 규칙 3의 예외이며, `record_decision` 이 `slice_id` · `vendor_id` 를 인자로 받기 때문이다.

조달이 기록보다 앞서는 것이 규칙 3과 충돌하지 않는 이유

규칙 3의 목적은 "판단했으나 실행하지 못한 경우를 남기는 것"이다. 조달은 판단의 일부로서 기록에 포함되므로 목적이 훼손되지 않는다.

```
조달 성공 → record_decision(slice_id="slice-...", vendor_id="vendor-1")
조달 실패 → record_decision(slice_id=null, rationale="procure rejected: capacity cap exceeded")
```

실패해도 `rationale` 에 남는다. 조달을 시도했다는 사실 자체가 판단의 증거이므로 기록되어야 한다.

`add_capacity` 는 반드시 `step()` (5) 전에 호출한다. 뒤로 가면 용량이 다음 스텝부터 반영되어 조달한 바로 그 스텝은 효과를 보지 못한다. 에이전트 입장에서 "샀는데 아무 일도 안 일어났다"가 되어 조달 학습이 왜곡된다.

3.2 `apply_allocation` 은 액추에이터다

현재 평활·클립·정규화가 `update_allocation_ml ( :401~407 )`과 `update_allocation_rule_based ( :459~462 )`에 중복되어 있다.

```
stability_factor = 0.7
new_allocation = 0.7 * self.allocation + 0.3 * target
new_allocation = np.clip(new_allocation, 0.1, 0.8)
new_allocation = new_allocation / np.sum(new_allocation)
```

이걸 정책에서 떼어 ④의 `apply_allocation()` 한 곳에 둔다. 세 가지 이득이 있다.

- **정책 비교가 공정해진다** — 모든 정책이 동일한 액추에이터를 통과한다. 정책은 "목표 배분"만 내고, 물리적 제약(급변 금지, 슬라이스당 10~80%)은 환경이 강제한다. 현실의 RAN 제약에 대응하므로 모델링으로도 타당하다.
- **에이전트가 제약을 학습할 수 있다** — 요청이 얼마나 깎였는지 `accepted / normalized / reason` 으로 돌려준다.
- **DQN 추가 시 재작성이 없다** — DQN은 이산 액션 27개를 배분 벡터로 바꾸기만 하면 된다.

```
apply_allocation(embb, urllc, mmtc) -> {
  "accepted": bool,
  "requested": {"embb": 0.20, "urllc": 0.70, "mmtc": 0.10},
  "normalized": {"embb": 0.34, "urllc": 0.49, "mmtc": 0.17}, # 평활 후
  "reason": "smoothed(0.7) + clipped to [0.1,0.8]"
}
```

`accepted=False` 는 입력이 비정상일 때만(음수, 합이 0, NaN). 정상 입력은 항상 수용하되 변형해서 알린다.

3.3 오케스트레이터는 고정 루프다 — LLM이 순서를 정하지 않는다

MCP는 도구를 줄 뿐 호출 순서를 강제하지 않는다. §3.1의 6단계를 LLM에게 맡기면 스텝당 6~10회 호출 × 120스텝 × 60회 실행 동안 반드시 다음이 일어난다.

- `report_outcome` 을 빠뜨린다 → 그 스텝이 채점되지 않는다 (§1.5 V2와 같은 증상)
- 순서가 뒤집힌다 → `add_capacity` 가 `step()` 뒤로 가면 조달이 무효가 된다 (§1.6 W1)
- 20~30스텝 뒤 드리프트한다 → 후반부 데이터가 전반부와 다른 절차로 생산된다

§8의 오류 규약이 잡는 것은 일부뿐이다. 대부분의 위반은 조용하다 — 호출하지 않은 것은 오류를 내지 않는다.

따라서 루프는 파이썬이 돌리고, LLM은 판단 지점에서만 호출한다.

```
for t in range(total_steps):
    obs = observe.get_observation()
    rel = feedback.get_reliability_table()

    j = ask_llm(obs, rel, recent)          # ← LLM은 여기서만
    # j.situation, j.confidence_situation
    # j.policy, j.procure_or_not, j.escalate_or_not

    if j.procure: ...                      # 2b
    audit.record_decision(...)             # 3 ← 파이썬이 보장
    observe.apply_allocation(...)          # 4
    observe.step()                         # 5
    feedback.report_outcome(...)           # 6
```

LLM이 답할 것은 네 가지뿐이다 — `situation` / 어느 정책 / 조달할까 / 사람을 부를까. §1.5에서 “*AGENT* 가 생산자인 값은 네 개뿐이어야 한다” 고 한 그 넷과 정확히 일치한다.

이것이 논문에도 유리하다. 자율성 주장은 판단에 관한 것이지 도구 호출 순서에 관한 것이 아니다. 루프를 고정하면

- 변수가 격리되어 비교군 간 차이가 전부 판단에서 나온다
- “에이전트가 루프를 잘못 돌아서 SLA가 나뻐다” 는 반론이 원천 차단된다
- 60회 실행이 동일한 절차로 생산된다

이 구조에서도 에이전트는 여전히 자율적이다. 무엇을 할지는 LLM이 정하고, 정한 것을 빠짐없이 실행하는 것만 파이썬이 보장한다. 사람으로 치면 절차를 지키는 것과 판단하는 것의 분리다.

3.4 컨텍스트는 스텝마다 재구성한다

120스텝 대화를 누적하면 후반에 100k 토큰을 넘는다. 비용·지연이 폭증하고, 모델이 20스텝 전의 낡은 관측에 주의를 뺏긴다.

매 스텝 프롬프트를 새로 만든다. 대화 이력을 이어붙이지 않는다.

시스템 프롬프트 (고정, minimal/advisory 2벌)

+ 현재 관측 1건

+ ④의 신뢰도 표

+ 최근 N스텝 요약 (N = 5, 한 줄씩: step/situation/policy/sla_met)

최근 이력을 **원본이 아니라 요약**으로 넣는 것이 핵심이다. `get_decisions(n=5)` 를 그대로 넣으면 관측이 통째로 달려와 5배가 된다 (TOOLS.md §9).

부수 효과로 프롬프트가 **재현 가능해진다**. 같은 스텝에서 같은 입력이면 같은 프롬프트가 만들어지므로, 프롬프트 해시를 `decisions.json` 에 기록해 사후 추적할 수 있다 (§9-9).

3.5 에스컬레이션의 조작적 정의

분리 설계서 §6의 "개입 1회"를 도구 수준으로 확정한다.

```
④.record_escalation(observation, reason, confidence) -> {"escalation_id": str}
```

- 이 도구를 **호출한 것 자체가 개입 1회**다. 별도의 사람 응답 대기는 없다.
- 호출 후 에이전트는 **rule_based + situation="normal" 로 진행한다**. 이것이 "사람이 올 때까지의 안전 기본 동작"이며, 사람의 개입 없이도 에피소드가 끝까지 돌아간다.
- 따라서 **모든 비교군이 동일한 스텝 수를 소화한다**. SLA 위반 비교가 공정해진다.

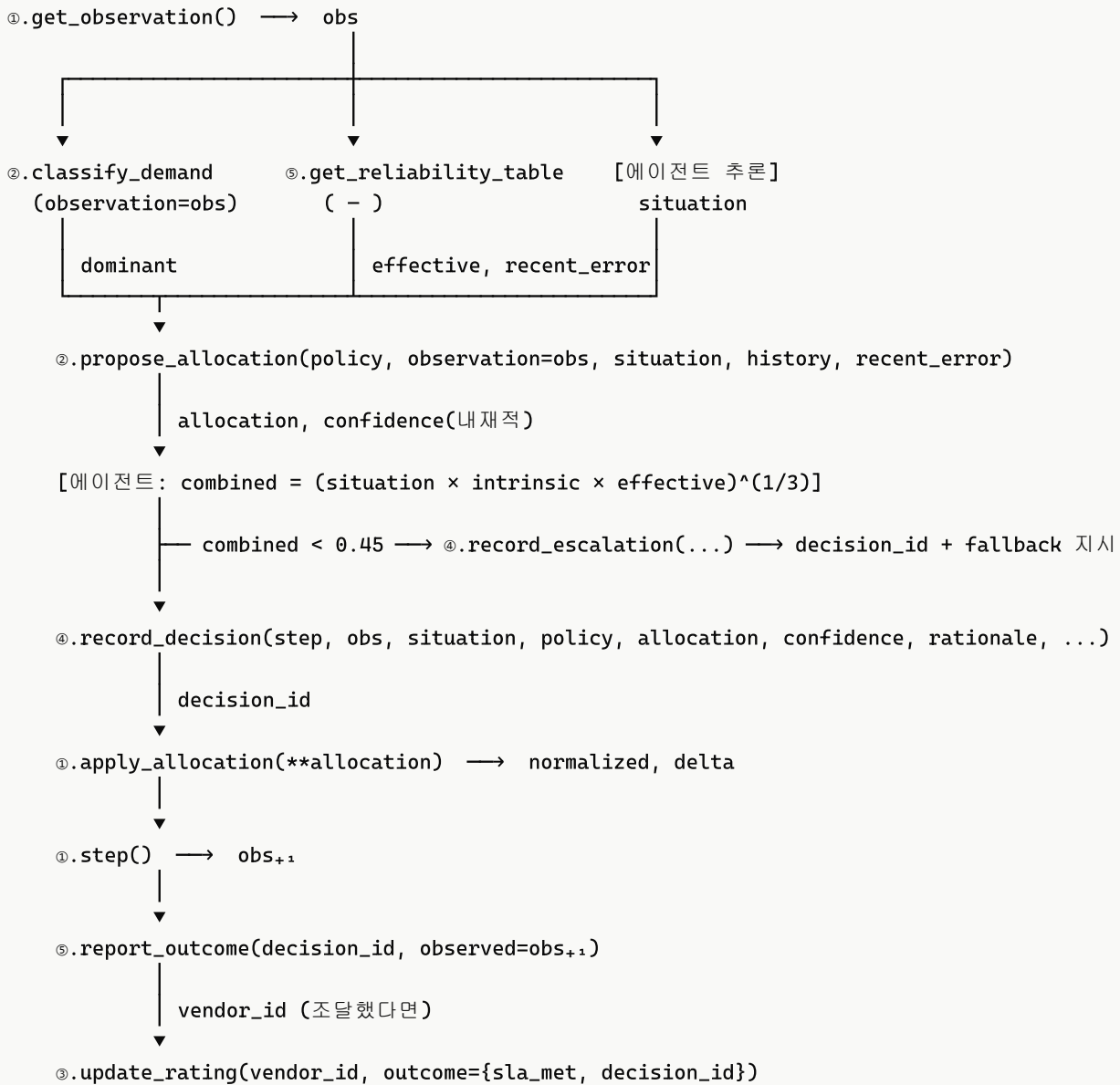
이 정의 덕분에 실험에 사람이 실제로 필요 없다. "**개입 횟수**"는 *사람을 부른 횟수이지 사람이 일한 시간이 아니다* — 이 점을 논문에 명시한다.

데이터 연쇄 — 어떤 출력이 어떤 입력이 되는가

공통 계약 · `claude/flow/data-chain.md`

서버끼리 직접 대화하지 않는다. 모든 값은 에이전트를 통과한다.

1스텝 흐름



조달 분기

`demand_pressure ≥ 1.0` 일 때만 의미가 있다. 배분 결정과 같은 스텝, `step()` 전에 일어난다.

```

obs.demand_pressure ≥ 1.0
    ↓
③.score_offerings(slice_type, qos) → vendor_id (1위)
    ↓
⑤.procure(vendor_id, slice_type, qos, duration_steps, current_step=obs.step)
    |
    capacity_gain, expires_at_step, slice_id
    ↓
①.add_capacity(slice_type, amount=capacity_gain, expires_at_step, slice_id)
    |
    demand_pressure (반영 후)
    ↓
[에이전트: 여전히 ≥ 1.0이면 추가 조달 또는 위반 감수]
slice_id · vendor_id → ④.record_decision에 함께 기록

```

③과 ①은 직접 대화하지 않는다. capacity_gain 을 에이전트가 옮겨 심는다. ⑤ → ③의 update_rating 중계, ⑤ → ②의 recent_error 중계와 같은 패턴이다.

출력 → 입력 대응표

생산	필드	소비	파라미터
①. get_observation	Observation 전체	②. propose_allocation	observation
		②. classify_demand	observation
		④. record_decision	observation
	demand_pressure	—	에이전트의 조달 판단 근거
①. get_history	HistoryBlock 전체	②. propose_allocation	history
①. step	observation	⑤. report_outcome	observed
②. propose_allocation	allocation	①. apply_allocation	embb / urllc / mmtc
	confidence	④. record_decision	confidence.intrinsic
	in_distribution	④. record_decision	in_distribution
②. classify_demand	출력 전체	④. record_decision	demand_class
②. compare_policies	탈락 후보 목록	④. record_decision	considered
③. score_offerings	vendor_id	③. explain_score · ③. procure	vendor_id
③. procure	slice_id	④. record_decision · ①. add_capacity	slice_id
	vendor_id	④. record_decision	vendor_id
	capacity_gain	①. add_capacity	amount
	expires_at_step	①. add_capacity	expires_at_step
④. record_decision	decision_id	⑤. report_outcome	decision_id
④. record_escalation	decision_id	⑤. report_outcome	decision_id
	fallback_allocation	①. apply_allocation	embb / urllc / mmtc
⑤. report_outcome	vendor_id , sla_met	③. update_rating	vendor_id , outcome
⑤. get_reliability_table	{policy}.effective	④. record_decision	confidence.empirical
	{policy}.recent_error	②. propose_allocation	recent_error
	테이블 전체	②. compare_policies	recent_errors
에이전트	run_id	①. reset · ④ 전체	run_id

에이전트가 만들어내는 값

어느 도구에서도 오지 않는다. LLM이 답하는 값은 정확히 넷이다. 어느 서버에도 이 코드가 없다.

값	쓰이는 곳	근거
situation	②. propose_allocation , ④. record_decision	관측(특히 트래픽 구성비)으로부터 추론
confidence.situation	④. record_decision	상황 판단 자체에 대한 확신. 어느 도구도 주지 않는다
정책 선택	②. propose_allocation 의 policy	신뢰도 비교
조달 여부	③. procure 호출 여부	demand_pressure + 비용 판단

나머지 둘은 파이썬이 기계적으로 계산한다. LLM에게 묻지 않는다.

값	계산
confidence.combined	(situation × intrinsic × empirical)^(1/3)
에스컬레이션 여부	combined < 0.45

check_contract.py 는 AGENT 생산 값이 정확히 이 넷인지 검사한다 — 늘면 판단이 서버로 샌 것이고, 줄면 기여가 사라진 것이다. (flow/loop.md — 루프는 파이썬이 돌리고 LLM은 판단 지점에서만 호출한다)

오류 규약

공통 계약 · `claude/flow/errors.md`

원칙: 에이전트가 판단으로 대응할 수 있는 것은 값으로, 코드 버그는 예외로.

값으로 돌려줄 것 vs 예외를 던질 것

상황	처리	이유
정책 사용 불가 (이력 부족, 모델 없음)	<code>status</code> 필드로 반환	에이전트의 판단 재료
배분 입력이 비정상	<code>accepted: false</code> 로 반환	에이전트가 재시도 가능
존재하지 않는 <code>vendor_id</code>	<code>error</code> 필드로 반환	오타 복구 가능
존재하지 않는 <code>decision_id</code>	<code>error</code> 필드로 반환	
용량 상한 초과 (<code>add_capacity</code>)	<code>accepted: false</code> 로 반환	에이전트가 다른 슬라이스를 택할 수 있음
<code>current_step</code> 이 과거 (<code>procure</code>)	<code>status: "rejected"</code> 로 반환	순서 실수. 복구 가능
잘못된 열거형 값	예외 (FastMCP가 스키마 검증)	계약 위반
파일 I/O 실패	예외	복구 불가

오류 메시지도 프롬프트의 일부다. 복구에 필요한 정보를 같이 준다.

```
// 존재하지 않는 벤더 - available을 같이 주면 에이전트가 한 번에 복구한다
{"error": "unknown_vendor", "vendor_id": "vendor-9",
 "available": ["vendor-1", "vendor-2", "vendor-3", "vendor-4", "vendor-5"]}
```

호출 순서 위반

위반	반환
<code>report_outcome</code> 을 <code>step()</code> 전에 호출	<code>{"error": "premature_scoring", "reason": "decision at step 12, observed step 12; expected >= 13"}</code>
<code>add_capacity</code> 를 <code>step()</code> 후에 호출	허용되지만 한 스텝 늦게 반영된다. 조달한 스텝은 효과를 못 본다
<code>procure</code> 없이 <code>add_capacity</code> 호출	허용. 용량은 늘지만 비용이 기록되지 않아 지표가 왜곡된다. 검사기가 경고
<code>apply_allocation</code> 없이 <code>step()</code> 두 번	허용. 직전 배분이 유지됨
<code>reset()</code> 없이 시작	허용. <code>seed=0</code> , <code>scenario="normal"</code> 로 자동 초기화

금지 필드 검사

공통 계약 · `claude/flow/forbidden.md`

`baseline` arm을 제외한 모든 실행에서 아래 문자열이 에이전트 컨텍스트에 0건이어야 한다. 도구 설명·반환값·오류 메시지 전부가 검사 대상이다. 1건이라도 나오면 그 실행은 폐기한다.

```
FORBIDDEN = ["is_emergency", "is_special_event", "is_iot_surge",
              "ground_truth", "perception_accuracy", "escalation_precision"]
```

출처	어디로만 나가나
<code>is_emergency</code> · <code>is_special_event</code> · <code>is_iot_surge</code>	①이 <code>runs/{run_id}/truth.jsonl</code> 에만 기록. 어떤 도구도 읽지 않는다
<code>perception_accuracy</code> · <code>escalation_precision</code>	<code>eval/score.py</code> 가 <code>truth.jsonl</code> 과 조인해 오프라인 계산. ④ <code>get_metrics</code> 는 반환하지 않는다

baseline arm의 예외

`baseline`만 `truth.jsonl`을 읽는다. 기존 시스템(CLI `--emergency`)의 재현이므로 정당하지만, 코드 경로를 물리적으로 분리한다. 같은 함수에 `if arm == "baseline"`으로 두면 실수로 다른 비교군에 샌다.

situation ↔ truth 매핑 규칙

`eval/score.py`가 `perception_accuracy`를 계산하려면 4값 열거형과 3개 `bool` 사이의 대응을 사전에 못박아야 한다. 채점 후에 정하면 유리한 쪽으로 해석할 수 있다.

truth	정답 situation
전부 <code>false</code>	<code>normal</code>
<code>is_emergency</code> 만 <code>true</code>	<code>emergency</code>
<code>is_special_event</code> 만 <code>true</code>	<code>special_event</code>
<code>is_iot_surge</code> 만 <code>true</code>	<code>iot_surge</code>
둘 이상 <code>true</code>	발생하지 않음

시나리오 정의상 세 플래그는 상호 배타적이다. `reset(scenario=...)`은 하나만 켜고, `mixed`의 전환 시점(`test_scenarios.py:92`)도 겹치지 않는다 — 스텝 20~50 `special_event`, 60~90 `emergency`, 100~120 `iot_surge`이며 그 사이는 전부 `normal`이다.

그래도 ①이 `truth.jsonl`을 쓸 때 상호 배타성을 `assert` 한다. 시나리오를 나중에 추가하다 겹치면 채점 기준이 조용히 무너진다.

전환 경계 처리

플래그가 바뀐 그 스텝의 결정은 정답 판정에서 제외한다. 에이전트는 `obs_t` 를 보고 판단하는데 `obs_t` 는 전환 직전의 트래픽을 반영하므로, 전환 스텝을 미탐으로 세면 원리적으로 맞힐 수 없는 것을 틀렸다고 세는 셈이다. `mixed` 120스텝에서 제외 대상은 5스텝이다.

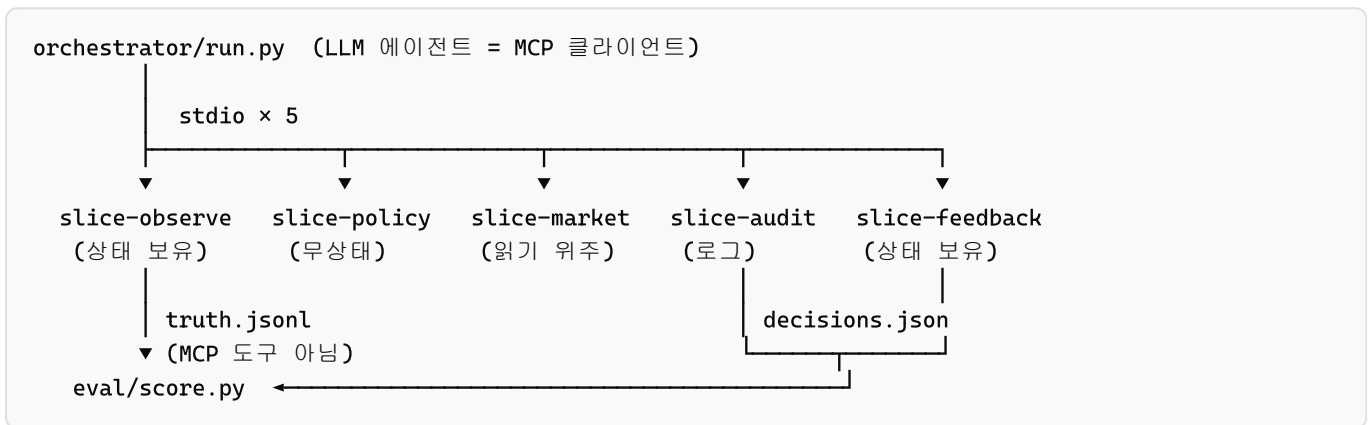
런타임 구조

공통 계약 · `claude/build/runtime.md`

프로세스 구성 · 저장소 레이아웃 · 기술 스택

2. 런타임 구조

프로세스 구성



- 전송: stdio. 단일 머신 연구 프로토타입이므로 HTTP 불필요. 서버당 독립 프로세스.
- ①만 에피소드 생명주기를 가진다. 나머지 넷은 프로세스가 죽어도 파일에서 복구된다.
- 동시 쓰기 없음: 에이전트가 도구를 순차 호출한다. 락을 두지 않는다 (분리 설계서 §2.1 승계).
- 단, 쓰기는 원자적으로 한다 — `tmp` 파일에 쓴 뒤 `os.replace()`. 중단 시 JSON 파일이 깨지는 것만 막으면 충분하다.

저장소 레이아웃

```
Build-A-Thon-SRM-main/
├── mcp/
│   ├── common/
│   │   ├── paths.py          # 경로 상수 한 곳
│   │   ├── schema.py        # Observation, Decision 등 pydantic 모델
│   │   └── store.py          # 원자적 JSON 읽기/쓰기
│   ├── observe/
│   │   ├── server.py         # ① FastMCP
│   │   └── env.py            # MLOrchestrator에서 추출한 순수 시뮬레이터
│   ├── policy/
│   │   ├── server.py         # ②
│   │   ├── descriptions.py   # 도구 설명 2벌 (§6.3)
│   │   ├── rule.py lstm.py dqn.py classify.py
│   ├── market/
│   │   ├── server.py         # ③
│   │   └── scoring.py        # engine.py에서 추출
│   ├── audit/
│   │   ├── server.py         # ④
│   │   └── metrics.py        # 관측 가능 지표만
│   ├── feedback/
│   │   ├── server.py         # ⑤
│   │   └── reliability.py     # EMA 갱신 (§6.2)
├── orchestrator/
│   ├── run.py                # 에이전트 루프 + 비교군 스위치
│   └── prompts/
├── eval/
│   ├── score.py              # 정답 조인 → 정답 의존 지표
│   └── plots.py
├── data/
│   ├── vendors.json          # 실행에 걸쳐 유지
│   └── vendors.json          # ⑥ 소유 (부트스트랩 시 모델 A 복사)
├── runs/{run_id}/
│   ├── decisions.json        # 실행마다 새로 (§5.0)
│   ├── decisions.json        # ④ 기록 · ⑤ 추가가입
│   ├── truth.jsonl           # ① 기록, 도구 미노출
│   ├── reliability.json      # ⑤ (cold 모드)
│   └── context/              # C의 누출 검사 덤프
├── tools/
│   └── step0_verify_models.py # 0단계
```

기존 코드는 건드리지 않는다. `mcp/` 는 `ml_orchestrator_demo.py` · `engine.py` 에서 로직을 추출(**copy-and-adapt**) 하지 저 파일들을 import 하지 않는다. 이유: 두 파일 다 최상단에서 무거운 부작용을 일으킨다 (`MLOrchestrator.__init__` 이 `results/` 폴더 생성, 모듈 레벨 `TF import`). 추출 대상은 §4에 위치를 명시했다.

기술 스택

항목	선택	근거
MCP 프레임워크	<code>fastmcp</code>	데코레이터 기반. 서버당 100줄 내외
Python	3.10	TF 2.15 호환 상한
TensorFlow	2.15.1 고정	Keras 2. SavedModel 직접 적재 (정정 I)
스키마	<code>pydantic</code> v2	FastMCP가 타입힌트에서 JSON Schema 자동 생성
가상환경	신규 생성	동봉된 <code>venv/</code> 는 Python 3.14 + 타 PC 절대경로. 사용 불가

공유 데이터 스키마

공통 계약 · claude/build/store.md

실행 생명주기 · decisions.json · reliability.json · truth.jsonl · vendors.json

5. 공유 데이터 스키마

5.0 실행 생명주기 — 무엇이 언제 초기화되는가

`reset()` 은 ①만 초기화한다. ②③④⑤의 상태는 그대로 남는다. 비교군 4 × 시나리오 5 × 시드 3 = 60회를 돌리는데 산출물 경로가 단일 파일이면 전부 덮어쓰거나 뒤섞인다.

실행 단위로 경로를 나눈다.

data/	← 실행에 걸쳐 유지
vendors.json	③ 소유. 레이팅이 누적된다
reliability.json	⑤ 소유. warm 모드에서만 사용
runs/{run_id}/	← 실행마다 새로
decisions.json	④ 생성 · ⑤ 추가가입
truth.jsonl	④ 기록. 도구 미노출
reliability.json	⑤. cold 모드에서만 사용
context/	C의 누출 검사용 덤프

서버	상태	실행 시작 시	근거
①	배분·용량·이력·리스	<code>reset()</code> 이 전부 초기화	에피소드가 곧 ①의 수명
②	없음 (모델만 상주)	초기화 불필요	무상태
③	vendors.json 의 rating	초기화 안 함 (기본)	레이팅 누적이 자기 개선의 증거
④	runs/{run_id}/decisions.json	첫 record_decision 이 파일 생성	별도 도구 불필요
⑤	reliability.json	모드에 따라 다름	아래

⑤와 ③의 초기화는 실험 변수다.

모드	⑤ reliability.json	③ vendors.json	보이는 것
cold	runs/{run_id}/ — 실행마다 새로	부트스트랩 재실행	단일 에피소드 내 적응
warm	data/ — 전체 누적	누적	장기 자기 개선 (주 결과)

모드는 ⑤·③ 서버 기동 시 환경변수(`SLICE_MEMORY_MODE`)로 정하고, decisions.json 의 config 에 기록한다. 60회 실행을 섞어 돌리면 안 된다 — warm 실행들 사이에 cold 실행이 끼면 누적 레이팅이 오염된다. 모드별로 묶어서 돌린다.

`run_id` 를 발급하는 것은 에이전트(C)다. 형식 `{arm}-{scenario}-s{seed}` . ①의 `reset()` 과 ④의 모든 기록에 같은 값이 들어가야 `truth.jsonl` 과 `decisions.json` 이 `step` 으로 조인된다 (§1.5 V3).

runs/{run_id}/decisions.json

④가 생성하고 ⑤가 같은 레코드에 덧붙인다 (분리 설계서 §2.1).

```
{
  "run_id": "exp-proposed-emergency-s0",
  "config": {"scenario": "emergency", "seed": 0, "arm": "proposed",
            "desc_mode": "minimal", "tau": 0.45},
  "decisions": [
    {
      "decision_id": "exp-proposed-emergency-s0-0012",
      "step": 12,
      "kind": "decision",
      "situation": "emergency",
      "chosen_policy": "rule_based",
      "allocation": {"embb": 0.20, "urllc": 0.70, "mmtc": 0.10},
      "confidence": {"intrinsic": 0.72, "empirical": 0.88, "combined": 0.80},
      "rationale": "URLLC util 1.52 > 1.2, eMBB 동반 상승 없음",
      "observation": { ... },

      "outcome": {                                // ← ⑤가 report_outcome에서 추가
        "sla_met": false,
        "error": 0.18,
        "vendor_id": null,
        "scored_at_step": 13
      }
    },
    { "decision_id": "...-0031", "kind": "escalation",
      "situation": "emergency", "reason": "combined 0.31 < tau 0.45",
      "confidence": {...} }
  ]
}
```

- `kind` 로 결정과 에스컬레이션을 한 배열에 둔다. 시간순 재구성이 쉬워진다.
- `config` 를 파일에 박는다. 비교군 4 × 시나리오 5 × 시드 N의 산출물이 섞이지 않는다.
- `outcome` 이 없는 레코드 = 마지막 스텝 또는 실행 실패. 집계에서 제외 (§3.1 규칙 2).

reliability.json (⑤ 소유)

```
{
  "rule_based": {"r": 0.91, "n": 120, "alpha": 0.2},
  "lstm_forecast": {"r": 0.84, "n": 95, "alpha": 0.2}
}
```

에피소드 간 유지 여부가 실험 설계 변수다. §6.2 참조.

runs/{run_id}/truth.jsonl (① 기록, 도구 미노출)

```
{"step": 12, "is_emergency": true, "is_special_event": false, "is_iot_surge": false}
```

data/vendors.json (③ 소유)

모델 A + regions. ⑤는 직접 쓰지 않는다 (분리 설계서 §2.1 승계).

3인 역할 분담 — 공통

공통 계약 · claude/team/roles.md

0. 병렬 개발의 전제

3명이 동시에 손대면 반드시 두 곳에서 막힙니다.

- A의 **Observation** 스키마를 B와 C가 동시에 소비한다. 이게 안 열면 아무도 시작할 수 없습니다.
- C는 ①~⑤가 다 있어야 루프를 돌릴 수 있다. 순진하게 짜면 C는 마지막 주까지 대기합니다.

두 문제를 Day 0 계약 동결과 C가 만드는 목 서버로 풀니다. §1과 §6이 이 문서의 핵심입니다.

1. Day 0 — 3명이 같이 앉아 끝낼 것 (반나절)

이걸 안 하고 흩어지면 3일 뒤에 합쳤을 때 아무것도 안 맞습니다.

1.1 저장소 정리 — 지금 당장 ⚠

현재 저장소에 병렬 개발을 방해하는 것이 둘 있습니다. 확인한 내용입니다.

(1) **.gitignore** 가 UTF-16으로 저장되어 있어 규칙이 먹지 않습니다.

```
$ file .gitignore
.gitignore: Unicode text, UTF-16, little-endian text, with CRLF
```

Git은 **.gitignore** 를 바이트로 읽습니다. UTF-16이면 모든 ASCII 문자 뒤에 `\x00` 이 붙어 패턴이 깨집니다. 실제로 **venv/** 가 무시되는 건 pip이 만든 **venv/.gitignore** 덕분이지 루트 규칙 때문이 아닙니다.

3명이 각자 실행하면 **data/*.json**, **results/**, **.venv310/** 이 전부 커밋 대상으로 잡혀 매 병합마다 충돌합니다. UTF-8로 다시 저장하고 런타임 산출물을 추가하세요.

```
# UTF-8(BOM 없음)로 재작성
printf '%s\n' '__pycache__/' '*.pyc' '*.egg-info/' 'venv/' '.venv*/' 'logs/' '*.env' \
'results/' 'runs/' 'data/decisions.json' 'data/reliability.json' \
'SRM-main-mcp/' > .gitignore
```

models/ 와 ***.h5** 는 넣지 마세요. 이미 추적 중이라 ignore 규칙이 적용되지 않고, 규칙만 보고 "가중치가 없구나" 오해하게 됩니다. 확인 결과 **가중치는 실물입니다** (LFS 스텝 아님, **variables.data-00000-of-00001** = 1.5 MB). B가 clone만 하면 바로 씁니다.

(2) 저장소 안에 저장소가 클론되어 있습니다.

```
srn-main_mcp/
└─ SRM-main-mcp/      ← 중첩 클론. git status에 ?? 로 뜸
    └─ .git
        └─ README.md
```

지우거나 밖으로 옮기세요. 그대로 두면 누군가 `git add .` 할 때 들어갑니다.

(3) 작업 기준 디렉터리 하나로 고정. `D:\pj\SRM-main\SRM-main` 은 옛 사본입니다. 3명 모두 `srn-main_mcp/` 만 씁니다.

1.2 계약 동결 — `mcp/common/`

A가 작성하고, 3명이 같이 읽고, 그 자리에서 확정합니다. 이후 변경은 PR + 3명 합의.

```
mcp/common/
├─ schema.py      # Observation, Decision, PolicyProposal, Outcome (pydantic)
├─ const.py       # 하드코딩 상수 전부
└─ store.py       # 원자적 JSON 읽기/쓰기
```

`const.py` 에 들어갈 값 — 흩어지면 3명이 각자 다른 값을 쓰게 됩니다.

상수	값	출처
THRESHOLDS	<code>{"embb": 0.9, "urllc": 1.2, "mmtc": 0.8}</code>	<code>ml_orchestrator_demo.py:174</code>
INIT_ALLOCATION	<code>{"embb": 0.4, "urllc": 0.4, "mmtc": 0.2}</code>	<code>:171</code>
STABILITY_FACTOR	<code>0.7</code>	<code>:458</code>
ALLOC_CLIP	<code>(0.1, 0.8)</code>	<code>:461</code>
MINUTES_PER_STEP	<code>15</code>	신규 (정정 F)
START_HOUR	<code>0</code>	신규. \$1.7에서 8 → 0으로 재조정
SEQUENCE_LENGTH	<code>10</code>	<code>:190</code>
TAU (에스컬레이션 문턱)	<code>0.45</code>	설계서 \$6.2
EMA_ALPHA	<code>0.2</code>	\$6.2
SHRINK_R0 , SHRINK_M	<code>0.5 , 5</code>	\$6.2
RATING_DELTA	<code>(+0.05, -0.20)</code>	\$4 ③
FEATURE_COLUMNS	11개 문자열 리스트	<code>:513~526</code>
CAPACITY_BASE	<code>1.6</code>	신규. 평시 여유 확보 (\$1.7)
CAPACITY_MAX	<code>2.6</code>	기본 + 조달 4회분 (\$1.7)
REFERENCE_BANDWIDTH	<code>{eMBB: 1100, URLLC: 500, mMTC: 120}</code>	<code>vendors.json</code> 중앙값 실측
GAIN_SCALE	<code>0.25</code>	신규. 튜닝 가능
COST_PER_STEP	<code>cost × MINUTES_PER_STEP / 60</code>	시간당 단가 → 스텝 환산 (W3)
MEMORY_MODE	<code>"warm"</code>	<code>cold / warm</code> . ③⑤의 실행 간 유지 여부 (W5)
SCENARIO_STEPS	<code>{normal:60, emergency:60, special_event:60, iot_surge:60, mixed:120}</code>	<code>test_scenarios.py:54</code>

TAU 는 실험 전에 확정하고 이후 절대 건드리지 않습니다. 조정하면 개입 횟수를 원하는 값으로 만들 수 있어 결과가 무의미해집니다.

1.3 파일 소유권 — 충돌 방지

한 파일은 한 사람만 수정합니다. 3인 병합 충돌의 90%가 여기서 예방됩니다.

경로	소유	비고
mcp/common/	A	Day 0 이후 변경은 3인 합의
mcp/observe/ , mcp/audit/	A	
eval/ , fixtures/ , tools/record_fixtures.py	A	
mcp/policy/ , mcp/market/ , mcp/feedback/	B	
tools/step0_verify_models.py , tools/bootstrap_vendors.py	B	
mcp/mock/	C	계약의 실행 가능 명세 (§6)
orchestrator/	C	
docs/	공용	섹션 단위로 나눠 쓰기
기존 코드 (ml_orchestrator_demo.py 등)	아무도 수정 안 함	읽기 전용. 로직은 복사해 가져감

마지막 줄이 중요합니다. 기존 파일을 고치면 3명이 같은 995줄 파일을 건드리게 됩니다. `mcp/` 는 **추출(copy-and-adapt)** 이지 import가 아닙니다.

1.4 브랜치 규칙

main

├─ feature/kim

├─ feature/lee

└─ feature/choi

← 통합 . 직접 커밋 금지

← A

← B

← C

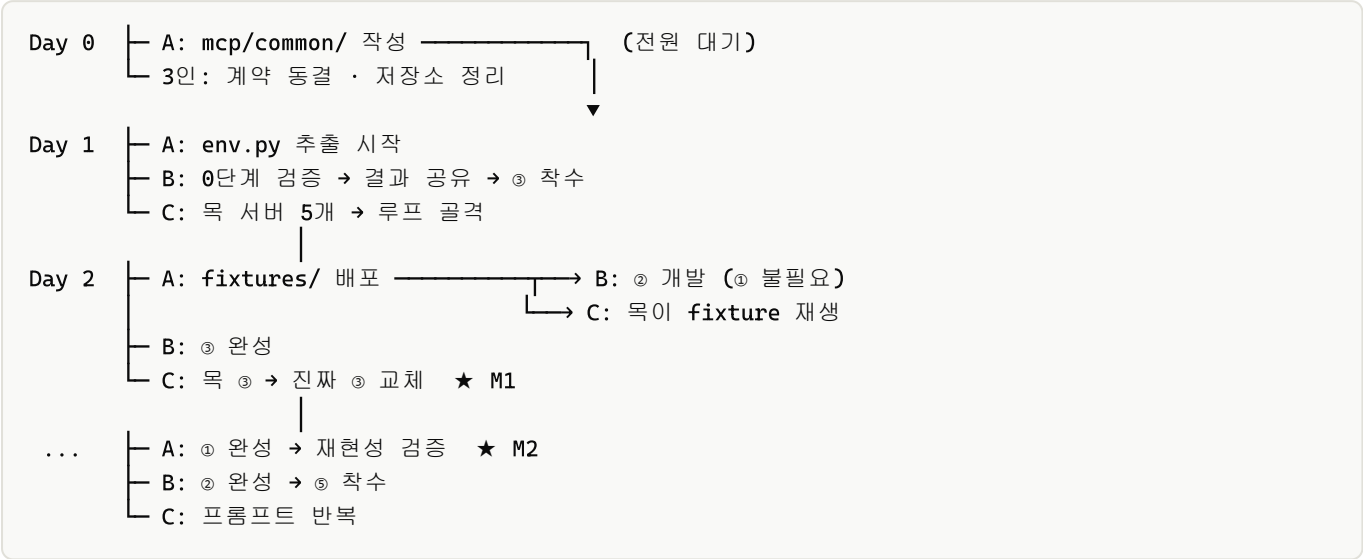
- 합류 지점(§7)마다 `main` 으로 PR. 그 사이에는 각자 브랜치에서 자유롭게.
- `main` 이 깨지면 3명이 동시에 막히므로, **PR 머지 전 최소 조건은 "내 서버가 단독 기동되고 도구 목록이 나온다"** 하나입니다. 테스트 통과까지 요구하면 초반에 진도가 안 나갑니다.

5. 왜 이렇게 묶었나

- A — 실험 인프라를 한 사람이.** `MLOrchestrator` 에서 `run()` 루프를 떼어내는 게 이 프로젝트에서 제일 까다롭습니다 (시뮬레이션·추론·시각화가 995줄에 뒤엉킴). ④ `audit`은 ①의 관측 스키마를 그대로 기록하니 같은 사람이 하면 손발이 맞습니다. 실험 하네스도 ④가 지표를 뽑으니 자연스럽게 따라옵니다.
- B — TensorFlow를 한 사람이.** ②와 ⑤ 모두 "정책"을 알아야 합니다. ⑤의 신뢰도 갱신은 ②가 어떤 정책을 노출하는지에 달려 있어서, 나누면 소통 비용만 늘어납니다. ③ `market`은 가볍게(`engine.py` 거의 그대로) 엮어도 부담이 적습니다.
- C — 판단 로직에 집중.** 오케스트레이터는 코드량은 적은데 반복 튜닝이 많습니다. 프롬프트 고치고 돌려보고 또 고치는 일이라, 서버 개발과 리듬이 다릅니다. 한 사람이 전담하는 게 맞습니다.

부하 균형: A와 B가 비슷하게 큼니다. C는 코드량이 가장 적은 대신 반복 횟수가 많고, 초반에 비는 시간을 목 서버 (C-0)와 누출 검사(C-4)로 채웁니다. 둘 다 C가 아니면 하기 어려운 일입니다.

6. 병렬화 장치 — 누가 누구를 기다리는가



막히지 않는 이유 세 가지

- 1. A의 common/ 이 Day 0에 나온다 — 그 전까지 아무도 못 움직이므로 최우선.
- 2. A의 fixtures/ 가 Day 2에 나온다 — B는 ① 없이 ②를, C는 ① 없이 프롬프트를 개발.
- 3. C의 목 서버가 Day 1에 나온다 — C는 A·B를 기다리지 않고, 동시에 계약 검증 장치가 생김.

교차 검증 — 자기 영역은 자기가 못 본다

검증 대상	검증자	방법
①④의 정답 누출 (정정 D)	C	컨텍스트 덤프 금지어 grep. A는 못 찾음
②의 situation 반영 (정정 E)	A	baseline 과 arm1 의 배분이 정답 일치 시 동일한가
②의 조용한 폴백 (정정 H)	A	④ policy_usage 와 ⑤ reliability 의 n 이 어긋나면 폴백 의심
①의 재현성 (정정 F)	A 자체	동일 시드 2회 → truth.jsonl 바이트 비교

7. 합류 지점

날짜 대신 조건으로 정의합니다. 각 지점에서 main 으로 PR.

	조건	확인 방법
M1 배선	C의 루프가 진짜 ③을 호출해 벤더 5개 점수 수신	목 하나가 진짜로 교체됨
M2 환경	① 동일 시드 2회 → <code>truth.jsonl</code> 동일. <code>fixtures/</code> 배포 완료	<code>cmp</code> 통과
M3 페루프	①②④⑤ 전부 진짜로 1스텝 왕복 + <code>report_outcome</code> 기록	<code>decisions.json</code> 에 <code>outcome</code> 필드 존재
M3.5 계약	<code>check_contract.py</code> 검사 4개 통과	값 유실 0건 (\$1.5)
M4 비교군	4개 arm이 <code>mixed</code> 120스텝 완주. 누출 검사 0건	<code>leak_check.py</code> 통과
M5 결과	60회 실행 + <code>score.py</code> 지표 산출 + 대표 그래프	개입 ↓ / SLA → 확인

M1을 가장 먼저, 가장 작게 만드세요. FastMCP 서버 하나, stdio 연결 하나, 도구 호출 하나가 왕복하면 나머지 20개는 같은 패턴의 반복입니다. ③이 중요해서가 아니라 배선 검증이 목적입니다.

M2에서 반드시 멈추고 확인하세요. 재현이 안 되는 채로 M3~M5를 쌓으면 마지막에 전부 다시 돌려야 합니다.

8. Day 0 체크리스트

셋이 같이 앉아 순서대로.

```
[ ] SRM-main-mcp/ 중첩 클론 제거
[ ] .gitignore UTF-8로 재작성 (models/, *.h5 는 넣지 말 것)
[ ] 작업 기준 디렉터리 srm-main_mcp/ 하나로 합의
[ ] docs/ 에 MCP-decomposition.md · MCP-design.md · ROLES.md 배치 후 커밋
[ ] A/B/C ⇄ kim/lee/choi 브랜치 매핑 확정
[ ] mcp/common/const.py 값 13개 확정 (§1.2) - 특히 TAU = 0.45
[ ] mcp/common/schema.py 확정 - Observation에 is_emergency 계열 부재 확인
[ ] docs/contract.json 초안 - 도구 21개의 requires/produces (C가 작성, 3인 검토)
[ ] run_id 형식 합의: {arm}-{scenario}-s{seed} (V3)
[ ] situation ⇄ truth 매핑·전환 경계 제외 규칙 확정 (V9) - 채점 전에 못 박을 것
[ ] 파일 소유권 표 합의 (§1.3)
[ ] 기존 코드는 아무도 수정하지 않는다 - 합의
[ ] B: Day 1 오전 0단계 착수, 결과 즉시 공유 - 합의
```

마지막 두 줄이 사고를 가장 많이 막습니다.

① `slice-observe` — 계약

담당 C · `claude/spec/observe.md`

시뮬레이션 환경 그 자체. 에이전트가 관측하고 행동을 가하는 대상. 설계 근거는 `rationale/observe.md`.

`get_observation()`

입력: 없음 · 출력: `Observation`

필드	타입	범위	의미
<code>step</code>	int	0 ~ <code>total_steps</code>	현재 스텝
<code>sim_time.hour_of_day</code>	int	0~23	가상 시계. 벽시계 아님
<code>sim_time.day_of_week</code>	int	0~6	0 = 월요일
<code>sim_time.is_weekend</code>	bool		<code>day_of_week >= 5</code>
<code>traffic</code>	<code>SliceTriple</code>	0.1 ~ 2.0	수요. <code>np.clip</code> 로 상하한 고정
<code>allocation</code>	<code>SliceTriple</code>	각 0.1~0.8, 합 1.0	현재 적용 중인 배분
<code>utilization</code>	<code>SliceTriple</code>	0 ~ 20	<code>traffic / (allocation × capacity)</code>
<code>capacity</code>	<code>SliceTriple</code>	1.6 ~ 2.6	슬라이스별 용량 배수. 기본 1.6, 조달로 증가, 만료로 감소
<code>thresholds</code>	<code>SliceTriple</code>	고정	<code>{0.9, 1.2, 0.8}</code> . 상수지만 매번 반환
<code>violations</code>	<code>SliceFlags</code>		<code>utilization > thresholds</code>
<code>client_count</code>	float	0 ~ 1	정규화된 단말 수
<code>bs_count</code>	float	0 ~ 1	정규화된 기지국 수
<code>demand_pressure</code>	float	0 ~ 2	$\sum_i \text{traffic} / (\theta \times \text{capacity})$. 1.0 초과면 재배분으로 해결 불가

`demand_pressure < 1.0` → 재배분만으로 해결 가능
`demand_pressure ≥ 1.0` → 용량을 늘리지 않으면 어딘가는 반드시 위반

서버는 숫자만 주고 "조달하라"는 조언은 하지 않는다.

실측(60스텝 × 시드 10): `normal` 5.6% · `emergency` 27.7% · `special_event` 34.3% · `iot_surge` 28.5%.
평시는 94%가 재배분으로 해결되고, 이벤트에서만 조달이 정답이 된다. (`rationale/environment.md`)

반환하지 않는 것 — `is_emergency` · `is_special_event` · `is_iiot_surge`. 환경 내부의 정답이며 `runs/{run_id}/truth.jsonl` 로만 나간다. 어떤 도구도 읽지 않는다. (`flow/forbidden.md`)

예시 (비상 상황 전개 중, 에이전트는 그 사실을 모름)

```
{
  "step": 12,
  "sim_time": {"hour_of_day": 3, "day_of_week": 0, "is_weekend": false},
  "traffic":    {"embb": 0.400, "urllc": 0.821, "mmtc": 0.325},
  "allocation": {"embb": 0.400, "urllc": 0.400, "mmtc": 0.200},
  "utilization": {"embb": 0.624, "urllc": 1.283, "mmtc": 1.015},
  "capacity":    {"embb": 1.600, "urllc": 1.600, "mmtc": 1.600},
  "thresholds":  {"embb": 0.900, "urllc": 1.200, "mmtc": 0.800},
  "violations":  {"embb": false, "urllc": true,  "mmtc": true},
  "client_count": 0.631,
  "bs_count": 0.482,
  "demand_pressure": 0.959
}
```

`scenario="emergency"`, `seed=0` 의 step 12 실측이다. 트래픽 구성비는 eMBB 26% · URLLC 53% · mMTC 21%로, 평시(44/33/22)와 비교하면 URLLC가 확연히 높다. `situation` 추론의 주된 근거는 절대량이 아니라 이 구성비다. `demand_pressure` `0.959 < 1.0` 이므로 이 스텝은 **재배분만으로 해결 가능하다**.

step(n)

파라미터	타입	필수	기본	범위	설명
<code>n</code>	int	아니오	1	1 ~ 10	전진할 스텝 수

출력 필드	타입	설명
<code>steps_advanced</code>	int	실제 전진한 수 (에피소드 끝에 걸리면 요청보다 적음)
<code>episode_done</code>	bool	<code>step >= total_steps</code>
<code>observation</code>	Observation	전진 후 관측

동작 순서 (고정): 완료된 조달 회수 → `capacity` 재계산 → `traffic_{t+1}` 생성 → `utilization_{t+1} = traffic_{t+1} / (allocation_t × capacity_{t+1})` → `truth.jsonl` append.

- 만료가 `traffic` 생성보다 먼저다.
- 이용률은 직전에 적용한 배분으로 계산된다. `step()` 직후의 관측 = 직전 결정의 성적표.
`report_outcome` 은 반드시 `step()` 이후.

```
{"steps_advanced": 1, "episode_done": false,
 "observation": {"step": 13, "traffic": {"embb": 0.571, "urllc": 0.595, "mmtc": 0.390}, ...}}
```

apply_allocation(embb, urllc, mmtc)

배분을 실제로 적용하는 액추에이터. 평활·클립·정규화가 전부 여기서 일어난다.

파라미터	타입	필수	범위
embb · urllc · mmtc	float	예	> 0

합이 1일 필요 없다. 서버가 정규화한다.

출력 필드	타입	설명
accepted	bool	입력이 유효했는가
requested	SliceTriple	정규화만 한 요청값
normalized	SliceTriple	실제 적용된 값 (평활 후)
delta	float	$L1(\text{requested}, \text{normalized}) / 2$. 얼마나 꺾였나
reason	string	어떤 변형이 가해졌는지

변형 파이프라인 (ml_orchestrator_demo.py:458~462)

- 정규화 $a / \sum a$
- 평활 $0.7 \times \text{현재배분} + 0.3 \times a$ ← STABILITY_FACTOR
- 클립 $\text{clip}(\cdot, 0.1, 0.8)$
- 재정규화 $a / \sum a$

```
// 요청
{"embb": 0.20, "urllc": 0.70, "mmtc": 0.10}
// 응답 - 평활 때문에 절반밖에 안 움직임
{"accepted": true,
 "requested": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
 "normalized": {"embb": 0.340, "urllc": 0.490, "mmtc": 0.170},
 "delta": 0.210,
 "reason": "smoothed(0.7); within clip [0.1, 0.8]}"
```

accepted: false 는 입력이 비정상일 때만 — 음수, 전부 0, NaN. 거부되어도 기존 배분은 유지된다.

```
{"accepted": false, "requested": null, "normalized": {"embb": 0.4, "urllc": 0.4, "mmtc": 0.2},
 "delta": 0.0, "reason": "rejected: negative value in urllc(-0.1); allocation unchanged"}
```

get_history(n)

LSTM 정책의 입력 시퀀스를 만든다.

파라미터	타입	필수	기본	범위
<code>n</code>	int	아니오	<code>10</code>	1 ~ 100

출력 필드	타입	설명
<code>n_requested</code>	int	요청한 수
<code>n_available</code>	int	실제 반환한 수. 초반에는 요청보다 적다
<code>columns</code>	<code>[string × 11]</code>	컬럼 이름. 순서 고정
<code>features</code>	<code>[[float × 11] × n_available]</code>	오래된 것 → 최신 순

`columns` 순서 — `create_feature_vector()` (:513~526)와 정확히 일치해야 한다. 매 호출 반환한다.

```
["traffic_load", "hour_of_day", "day_of_week",
 "embb_allocation", "urllc_allocation", "mmtc_allocation",
 "embb_utilization", "urllc_utilization", "mmtc_utilization",
 "client_count", "bs_count"]
```

- 피쳐는 **11차원 고정**. `capacity` 는 포함되지 않는다 (재학습 없이는 추가 불가). 조달 직후 `in_distribution: false` 가 뜰 수 있다 — 버그가 아니다. ([rationale/observe.md](#))
- `n_available < 10` 이면 `lstm_forecast` 를 쓸 수 없다. ②가 `status: "unavailable"` 을 반환한다.

```
{ "n_requested": 10, "n_available": 4,
  "columns": ["traffic_load", "hour_of_day", ...],
  "features": [
    [0.4893, 0.0417, 0.0, 0.400, 0.400, 0.200, 0.605, 1.196, 0.948, 0.610, 0.470],
    [0.5020, 0.0833, 0.0, 0.400, 0.400, 0.200, 0.613, 1.231, 0.971, 0.618, 0.475],
    [0.5157, 0.0833, 0.0, 0.400, 0.400, 0.200, 0.619, 1.258, 0.994, 0.625, 0.479],
    [0.5153, 0.1250, 0.0, 0.400, 0.400, 0.200, 0.624, 1.283, 1.015, 0.631, 0.482]] }
```

reset(run_id, scenario, seed)

파라미터	타입	필수	기본	허용값
<code>run_id</code>	string	예	—	이 실행의 식별자. <code>truth.jsonl</code> 의 모든 줄에 박힌다. ④에도 같은 값을 넘긴다
<code>scenario</code>	string	아니오	<code>"normal"</code>	<code>normal</code> · <code>emergency</code> · <code>special_event</code> · <code>iot_surge</code> · <code>mixed</code>
<code>seed</code>	int	아니오	<code>0</code>	임의 정수

출력 필드	타입	설명
<code>run_id</code>	string	적용된 실행 식별자
<code>scenario</code>	string	적용된 시나리오
<code>capacity</code>	<code>SliceTriple</code>	항상 <code>{1.6, 1.6, 1.6}</code> (<code>CAPACITY_BASE</code>). 활성 조달 전부 회수
<code>seed</code>	int	적용된 시드
<code>total_steps</code>	int	이 에피소드의 길이
<code>observation</code>	<code>Observation</code>	<code>step: 0</code>

시나리오별 길이 (`test_scenarios.py:54-102`)

시나리오	<code>total_steps</code>	전개
<code>normal</code>	60	평시 유지
<code>emergency</code>	60	URLLC ×2.0 유지
<code>special_event</code>	60	eMBB ×1.5 유지
<code>iot_surge</code>	60	mMTC ×1.8 유지
<code>mixed</code>	120	20: 이벤트 on / 50: off / 60: 비상 on / 90: off / 100: IoT on

- 동일 `seed` → 동일 전개. 전용 RNG(`np.random.default_rng(seed)`)와 가상 시계.
- `reset()` 은 ①만 초기화한다. ②③④⑤의 상태는 남는다.
- `truth.jsonl` 각 줄: `{"run_id": "...", "step": 12, "is_emergency": true, "is_special_event": false, "is_iot_surge": false}`

```
{"run_id": "exp-proposed-emergency-s0", "scenario": "emergency", "seed": 0, "total_steps": 60,
 "observation": {"step": 0, "allocation": {"embb": 0.4, "urllc": 0.4, "mmtc": 0.2}, ...}}
```

`add_capacity(slice_type, amount, expires_at_step, slice_id)`

조달한 슬라이스를 환경에 실제로 반영한다. ③의 `procure()` 출력을 에이전트가 중계한다.

파라미터	타입	필수	범위	출처
<code>slice_type</code>	<code>SliceType</code>	예		
<code>amount</code>	float	예	$0 < x \leq 0.5$	③ <code>procure().capacity_gain</code>
<code>expires_at_step</code>	int	예	> 현재 step	③ <code>procure().expires_at_step</code>
<code>slice_id</code>	string	예		③ <code>procure().slice_id</code>

출력: CapacityState

필드	타입	설명
accepted	bool	상한 초과 시 false
capacity	SliceTriple	반영 후 용량
demand_pressure	float	반영 후 압력. 즉시 효과 확인용
active_leases	[{slice_id, slice_type, amount, expires_at_step}]	활성 조달 목록
reason	string null	거부 사유

- **용량 상한**: 슬라이스당 2.6 (기본 1.6 + 조달 4회분). 초과 요청은 거부.
- 상한을 3.6 으로 올리면 모든 스텝이 조달로 해소되어 에스컬레이션이 무의미해진다. 2.6 에서 막는다.
- **거부되어도 ③의 procure() 는 이미 일어났다.** 비용은 청구되고 용량은 안 늘어난다. 호출 전에 Observation.capacity 로 상한을 확인할 것.

```
// 요청
{"slice_type": "URLLC", "amount": 0.25, "expires_at_step": 22, "slice_id": "slice-urllc-0012-v1"}
// 응답 - 압력 0.959 → 0.901. 조달 1회가 약 6%를 덜어낸다
{"accepted": true,
 "capacity": {"embb": 1.600, "urllc": 1.850, "mmtc": 1.600},
 "demand_pressure": 0.901,
 "active_leases": [{"slice_id": "slice-urllc-0012-v1", "slice_type": "URLLC",
                    "amount": 0.25, "expires_at_step": 22}],
 "reason": null}
// 거부
{"accepted": false, "capacity": {"embb": 1.6, "urllc": 2.6, "mmtc": 1.6},
 "demand_pressure": 0.842, "active_leases": [...],
 "reason": "capacity_cap_exceeded: urllc 2.60 + 0.25 > 2.6"}
```

② `slice-policy` — 계약

담당 C · `claude/spec/policy.md`

무상태. 배분 정책 여러 개를 나란히 노출한다. 설계 근거는 `rationale/policy.md`.

`list_policies()`

입력: 없음 · 출력: `[PolicyInfo]`

필드	타입	설명
<code>name</code>	<code>PolicyName</code>	
<code>description</code>	<code>string</code>	<code>SLICE_DESC_MODE</code> 에 따라 달라짐 (<code>minimal</code> / <code>advisory</code>)
<code>requires</code>	<code>string null</code>	전제조건
<code>available</code>	<code>bool</code>	지금 호출 가능한가
<code>unavailable_reason</code>	<code>string null</code>	불가 사유

TensorFlow 적재에 실패하면 서버는 죽지 않고 해당 정책만 `available: false` 로 내린다.

```
[
  {
    "name": "rule_based",
    "description": "임계값 기반 배분. 상황 라벨을 입력으로 받는다.",
    "requires": null,
    "available": true,
    "unavailable_reason": null,
  },
  {
    "name": "lstm_forecast",
    "description": "시계열 모델 기반 배분. 관측 이력 10스텝 필요.",
    "requires": "history >= 10",
    "available": true,
    "unavailable_reason": null,
  },
  {
    "name": "dqn",
    "description": "강화학습 정책 기반 배분.",
    "requires": "trained weights",
    "available": false,
    "unavailable_reason": "no trained weights"
  }
]
```

`propose_allocation(policy, observation, situation, history, recent_error)`

파라미터	타입	필수	기본	출처
<code>policy</code>	<code>PolicyName</code>	예	—	
<code>observation</code>	<code>Observation</code>	예	—	①. <code>get_observation()</code> 그대로
<code>situation</code>	<code>Situation</code>	예	없음	에이전트가 추론한 상황
<code>history</code>	<code>HistoryBlock</code>	아니오	<code>null</code>	①. <code>get_history()</code> 그대로
<code>recent_error</code>	<code>float</code>	아니오	<code>null</code>	⑤. <code>get_reliability_table()</code> 의 <code>{policy}.recent_error</code>

- **situation** 에 기본값을 두지 않는다. 모든 정책이 필수로 받는다 (배분에 영향은 **rule_based** 만).
- **recent_error** 는 에이전트가 ⑤에서 ②로 중계한다. ②는 무상태라 ⑤를 조회할 수 없다. **null** 이면 보수적 기본값 **0.5** 를 쓰고 **rationale** 에 그 사실을 적는다.

출력: **PolicyProposal**

필드	타입	설명
policy	PolicyName	요청한 정책. 절대 다른 값으로 바꾸지 않는다
allocation	SliceTriple null	목표 배분. 실패 시 null
confidence	float	0~1. 내재적 신뢰도 — 이번 입력에 대한 확신
in_distribution	bool	학습 분포 안인가
status	Status	ok · unavailable · error
reason	string null	status != ok 일 때의 사유
rationale	string	사람이 읽는 근거

폴백 금지 ⚠ — 실패 시 다른 정책을 조용히 호출하지 않는다. **allocation: null** + **status** 로 반환한다. 원본 **update_allocation_ml()** (:341, :414, :419)의 폴백을 옮기지 말 것. 대체 정책 선택은 에이전트의 일이다.

confidence 산출

정책	식
rule_based	$0.50 + 0.30 \times \min(1, \min_i u_i - \theta_i / \theta_i)$
lstm_forecast	$\exp(-3 \cdot \bar{e})$, $\bar{e}$ = recent_error (최근 5회 예측 오차의 EMA, ⑤가 계산)

recent_error 가 **null** 이면 **0.5** 를 쓴다.

예시 1 — 성공

```
// 요청
{"policy": "rule_based",
 "observation": {"step": 12, "utilization": {"embb": 0.624, "urllc": 1.283, "mmtc": 1.015}, ...},
 "situation": "emergency"}
// 응답
{"policy": "rule_based",
 "allocation": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
 "confidence": 0.521,
 "in_distribution": true,
 "status": "ok",
 "reason": null,
 "rationale": "situation=emergency → URLLC 우선 목표 [0.2, 0.7, 0.1]. 여유가 가장 작은 것은 URLLC(1.283/1.2, 7% 초과)라 분기가 자의적 → confidence 0.521."}
```

예시 2 — 이력 부족

```
{
  "policy": "lstm_forecast",
  "allocation": null,
  "confidence": 0.0,
  "in_distribution": false,
  "status": "unavailable",
  "reason": "history_insufficient: 4 < 10",
  "rationale": "시퀀스 길이 미달로 추론하지 않음."
}
```

예시 3 — 모델 적재 실패

```
{
  "policy": "lstm_forecast",
  "allocation": null,
  "confidence": 0.0,
  "in_distribution": false,
  "status": "error",
  "reason": "model_load_failed: SavedModel requires Keras 2 (tensorflow==2.15.1)",
  "rationale": "모델을 적재하지 못해 추론 불가."
}
```

compare_policies(observation, situation, history, recent_errors)

propose_allocation 에서 policy 를 빼고, recent_error 를 정책별 딕셔너리로 바꾼 것.

파라미터	타입	필수	출처
observation	Observation	예	
situation	Situation	예	
history	HistoryBlock	아니오	
recent_errors	{PolicyName: float}	아니오	⑤. get_reliability_table() 을 그대로 넘긴다

출력: [PolicyProposal] — 사용 가능 여부와 무관하게 전 정책을 반환한다.

```
[
  {
    "policy": "rule_based",
    "allocation": {
      "embb": 0.20,
      "urllc": 0.70,
      "mmtc": 0.10
    },
    "confidence": 0.521,
    "status": "ok",
    ...
  },
  {
    "policy": "lstm_forecast",
    "allocation": null,
    "confidence": 0.0,
    "status": "unavailable",
    "reason": "history_insufficient: 4 < 10",
    ...
  },
  {
    "policy": "dqn",
    "allocation": null,
    "confidence": 0.0,
    "status": "unavailable",
    "reason": "no trained weights",
    ...
  }
]
```

호출 비용 주의 — 반환량이 propose_allocation 의 3배. 정책 전환을 고민할 때만 쓰도록 프롬프트에서 유도한다.

classify_demand(observation)

학습된 분류기(TrafficClassifier, 11 → 3 softmax)로 어떤 수요가 지배적인지만 알려준다.

파라미터	타입	필수
observation	Observation	예

출력 필드	타입	설명
dominant	SliceType	최대 확률 클래스
probabilities	{eMBB, URLLC, mMTC: float}	합 1.0
margin	float	1등 - 2등 확률차
available	bool	모델 적재 여부

```
{
  "dominant": "URLLC",
  "probabilities": {
    "eMBB": 0.281,
    "URLLC": 0.644,
    "mMTC": 0.075
  },
  "margin": 0.363,
  "available": true
}
```

이 도구는 "비상 상황인가"를 말하지 않는다. 상황 판단은 에이전트의 몫이다.

③ `slice-market` — 계약

담당 C · `claude/spec/market.md`

외부 벤더 슬라이스 마켓. 설계 근거는 `rationale/market.md`.

qos_requirements 스펙 ⚠

③의 도구 3개(`score_offerings` · `explain_score` · `procure`)가 공통으로 받는다. 코드가 실제로 읽는 키는 이것뿐이다 (`engine.py:339`, `:368`, `:398`):

키	타입	기본값	단위	의미
<code>latency</code>	float	<code>50</code>	ms	요구 지연 상한
<code>bandwidth</code>	float	<code>100</code>	Mbps	요구 대역 하한
<code>reliability</code>	float	<code>99.0</code>	%	요구 신뢰도 하한
<code>advanced_params</code>	object	<code>{}</code>		선택. 1차 범위에서는 쓰지 않음

키 이름에 단위를 붙이지 않는다. `latency_ms` 가 아니라 `latency` 다. 혼동하면 전부 기본값으로 떨어져 모든 벤더가 같은 점수를 받는다.

`advanced_params` 허용 키: `price_sensitivity` (`high|medium|low` → `price_weight` 0.3/0.2/0.1) · `deterministic` · `missionCritical` · `maxPacketSize` · `groupCommunication` · `availability`

list_offerings(slice_type, region)

파라미터	타입	필수	기본	설명
<code>slice_type</code>	<code>SliceType</code>	아니오	<code>null</code>	필터. <code>null</code> 이면 전부
<code>region</code>	string	아니오	<code>null</code>	필터

출력: `[Offering]` — 벤더 5곳 × 슬라이스 3타입. 원본 `data/vendors.json`.

필드	타입	단위
vendor_id · name	string	
slice_type	SliceType	
latency	float	ms
bandwidth	float	Mbps
reliability	float	%
cost	float	시간당
regions	[string]	
rating	float	1.0 ~ 5.0

```
// 요청
{"slice_type": "URLLC"}
// 응답 (실측값, 일부)
[
  {"vendor_id": "vendor-1", "name": "TelcoNet Solutions", "slice_type": "URLLC",
   "latency": 1.0, "bandwidth": 500.0, "reliability": 99.999, "cost": 250.0,
   "regions": ["us-east"], "rating": 4.8},
  {"vendor_id": "vendor-2", "name": "GlobalConnect 5G", "slice_type": "URLLC",
   "latency": 2.0, "bandwidth": 400.0, "reliability": 99.99, "cost": 220.0,
   "regions": ["eu-west"], "rating": 4.5},
  {"vendor_id": "vendor-3", "name": "NextGen Networks", "slice_type": "URLLC",
   "latency": 0.5, "bandwidth": 600.0, "reliability": 99.9995, "cost": 300.0,
   "regions": ["ap-northeast"], "rating": 4.9}
]
```

score_offerings(slice_type, qos_requirements)

파라미터	타입	필수
slice_type	SliceType	예
qos_requirements	object	예

출력: [ScoredOffering] — 점수 내림차순

필드	타입	범위
rank	int	1부터
vendor_id · name	string	
score	float	0 ~ 100
rating	float	1.0 ~ 5.0
cost	float	

점수 공식 (engine.py:719~790 , score_vendor_offering())

```
qos_score = Σ(scorei × weighti) / Σ(weighti)      # 존재하는 기준만, 정규화
reputation = rating / 5.0                             # 가중치 0.2 (URLLC) / 0.1 (그 외)
price      = _calculate_price_score(cost, slice_type)  # 가중치 0.2 (기본)
qos_weight = 1.0 - (reputation_weight + price_weight)
total      = (qos_score × qos_weight + reputation × rep_w + price × price_w) × 100
```

rating 이 총점의 10~20%를 차지한다. ⑤의 피드백이 update_rating 으로 여기에 들어와 순위를 바꾼다.

```
// 요청
{"slice_type": "URLLC", "qos_requirements": {"latency": 1.0, "bandwidth": 400, "reliability": 99.99}}
// 응답 (engine.py 실행 실측값)
[
  {"rank": 1, "vendor_id": "vendor-1", "name": "TelcoNet Solutions", "score": 99.20, "rating": 4.8, "cost": 250.0},
  {"rank": 2, "vendor_id": "vendor-5", "name": "AsiaPacific Telecom", "score": 98.00, "rating": 4.7, "cost": 260.0},
  {"rank": 3, "vendor_id": "vendor-3", "name": "NextGen Networks", "score": 97.60, "rating": 4.9, "cost": 300.0},
  {"rank": 4, "vendor_id": "vendor-4", "name": "EuroSlice Providers", "score": 91.76, "rating": 4.6, "cost": 240.0},
  {"rank": 5, "vendor_id": "vendor-2", "name": "GlobalConnect 5G", "score": 86.20, "rating": 4.5, "cost": 220.0}
]
```

explain_score(vendor_id, slice_type, qos_requirements)

파라미터	타입	필수
vendor_id	string	예
slice_type	SliceType	예
qos_requirements	object	예

출력: ScoreBreakdown

필드	타입	설명
total	float	score_offerings 의 score 와 반드시 일치
criteria_scores	{criterion: float}	기준별 원점수 0~1
weights	{criterion: float}	적용된 가중치
contributions	{criterion: float}	총점 100 기준 기여분
explanation	string	사람이 읽는 요약

구현 규칙 ⚠️ (정정 J) - 기존 `get_score_breakdown()` (engine.py:819)을 호출하지 않는다. `rating` 대신 존재하지 않는 `reputation_score` 키를 읽어 총점이 어긋난다 (99.20 vs 92.00). - `score_vendor_offering()` 의 실제 계산을 분해해 새로 구현한다 (약 30줄). - `neural_network` 키(601바이트, UI용 장식)는 반환하지 않는다.

```
{
  "total": 99.20,
  "criteria_scores": {"latency": 1.00, "bandwidth": 1.00, "reliability": 1.00,
    "reputation": 0.96, "price": 1.00},
  "weights": {"latency": 0.35, "bandwidth": 0.10, "reliability": 0.25,
    "reputation": 0.20, "price": 0.20},
  "contributions": {"latency": 30.00, "bandwidth": 8.57, "reliability": 21.43,
    "reputation": 19.20, "price": 20.00},
  "explanation": "지연 1.0ms가 요구 1.0ms를 충족(1.00). 평판 4.8/5.0이 총점의 19.2점을 기여. 가격 250이 URLLC 기준가와 동일해 감점 없음."
}
```

procure(vendor_id, slice_type, qos_requirements, duration_steps, current_step)

파라미터	타입	필수	범위	설명
vendor_id	string	예		
slice_type	SliceType	예		
qos_requirements	object	예		
duration_steps	int	예	1 ~ 60	시간이 아니라 스텝
current_step	int	예	≥ 0	observation.step. ③은 시뮬레이션 시각을 모른다

비용 환산 — `vendors.json` 의 `cost` 는 시간당, `duration_steps` 는 스텝. `MINUTES_PER_STEP = 15`.

```
cost_total = cost × duration_steps × (MINUTES_PER_STEP / 60)
            = 250 × 10 × 0.25 = 625.0
```

용량 환산 — 상수는 `const.py`.

`capacity_gain` = (`offering.bandwidth` / `REFERENCE_BANDWIDTH[slice_type]`) × `GAIN_SCALE`
`expires_at_step` = `current_step` + `duration_steps`

상수	값
<code>REFERENCE_BANDWIDTH</code>	{eMBB: 1100, URLLC: 500, mMTC: 120} (벤더별 슬라이스 대역폭 중앙값)
<code>GAIN_SCALE</code>	0.25
<code>CAPACITY_BASE</code> / <code>CAPACITY_MAX</code>	1.6 / 2.6 (슬라이스당, ①이 강제)

벤더 (URLLC)	대역폭	<code>capacity_gain</code>
GlobalConnect 5G	400	0.20
TelcoNet Solutions	500	0.25
NextGen Networks	600	0.30

출력 필드	타입	설명
<code>slice_id</code>	string	조달된 슬라이스 식별자. ④ <code>record_decision.slice_id</code> 에 기록
<code>status</code>	string	active · rejected
<code>vendor_id</code>	string	
<code>cost_total</code>	float	위 환산식
<code>capacity_gain</code>	float	①의 <code>add_capacity(amount=...)</code> 에 그대로 넣는다
<code>expires_at_step</code>	int	만료 스텝
<code>reason</code>	string null	거부 사유

```
// 요청
{"vendor_id": "vendor-1", "slice_type": "URLLC",
 "qos_requirements": {"latency": 1.0, "bandwidth": 400, "reliability": 99.99},
 "duration_steps": 10, "current_step": 12}
// 응답
{"slice_id": "slice-urllc-0012-v1", "status": "active", "vendor_id": "vendor-1",
 "cost_total": 625.0, "capacity_gain": 0.250, "expires_at_step": 22, "reason": null}
```

비용은 제약이 아니다. 예산 없음. ④가 `procurement_cost_total` 을 집계해 논문 지표로 보고한다.

`update_rating(vendor_id, outcome)`

⑤의 `report_outcome` 결과를 에이전트가 증계해 호출한다.

파라미터	타입	필수	출처
<code>vendor_id</code>	string	예	⑤ <code>report_outcome().vendor_id</code>
<code>outcome.sla_met</code>	bool	예	⑤ <code>report_outcome().sla_met</code>
<code>outcome.decision_id</code>	string	예	추적용

출력 필드	타입
<code>vendor_id</code>	string
<code>rating_before</code> · <code>rating_after</code>	float
<code>delta</code>	float

갱신식: `rating` $\leftarrow$ `clip(rating +  $\delta$ , 1.0, 5.0)`, $\delta = +0.05$ (충족) / `-0.20` (위반). `vendors.json` 에 반영.

```
// 요청
{"vendor_id": "vendor-1", "outcome": {"sla_met": false, "decision_id": "exp-proposed-emergency-s0-0012"}}
// 응답
{"vendor_id": "vendor-1", "rating_before": 4.80, "rating_after": 4.60, "delta": -0.20}
```

④ `slice-audit` — 계약

담당 C · `claude/spec/audit.md`

판단 기록과 지표 집계. 설계 근거는 `rationale/audit.md`. 기록 파일: `runs/{run_id}/decisions.json`.

`record_decision(...)`

판단을 실행(`apply_allocation`)보다 먼저 기록한다.

파라미터	타입	필수	출처
<code>step</code>	<code>int</code>	예	<code>observation.step</code>
<code>observation</code>	<code>Observation</code>	예	판단의 근거가 된 관측 (<code>obs_t</code>)
<code>situation</code>	<code>Situation</code>	예	에이전트의 상황 판단. <code>perception_accuracy</code> 의 입력
<code>chosen_policy</code>	<code>PolicyName</code>	예	
<code>allocation</code>	<code>SliceTriple</code>	예	②가 제안한 값 (적용 전)
<code>confidence.situation</code>	<code>float</code>	예	상황 판단에 대한 확신. 에이전트가 직접 낸다
<code>confidence.intrinsic</code>	<code>float</code>	예	②의 <code>confidence</code> (정책에 대한 확신)
<code>confidence.empirical</code>	<code>float</code>	예	⑤의 <code>effective</code>
<code>confidence.combined</code>	<code>float</code>	예	$(\text{situation} \times \text{intrinsic} \times \text{empirical})^{(1/3)}$
<code>rationale</code>	<code>string</code>	예	판단 근거
<code>slice_id</code>	<code>string</code>	아니오	조달했다면 ③의 <code>slice_id</code>
<code>vendor_id</code>	<code>string</code>	아니오	조달했다면 ③의 <code>vendor_id</code> . ⑤가 이걸 읽어 에이전트에 돌려준다
<code>in_distribution</code>	<code>bool</code>	아니오	②의 <code>PolicyProposal.in_distribution</code>
<code>demand_class</code>	<code>object</code>	아니오	②의 <code>classify_demand()</code> 출력
<code>considered</code>	<code>[{policy, confidence, status}]</code>	아니오	<code>compare_policies</code> 를 썼다면 탈락한 후보들

출력

```
{"decision_id": "exp-proposed-emergency-s0-0012", "recorded_at_step": 12}
```

`decision_id` 형식: `{run_id}-{step:04d}`. 정렬 가능하고 `truth.jsonl`과 `step`으로 조인된다.

```
// 요청 예시
{
  "step": 12,
  "observation": {"step": 12, "utilization": {"embb": 0.624, "urllc": 1.283, "mmtc": 1.015}, ...},
  "situation": "emergency",
  "chosen_policy": "rule_based",
  "allocation": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
  "confidence": {"situation": 0.70, "intrinsic": 0.521, "empirical": 0.880, "combined": 0.685},
  "rationale": "URLLC 구성비 53% (평시 33%), 새벽 3시인데 URLLC 지배적. 이용률 1.283 (임계의 107%). LS
  TM은 이력 부족으로 사용 불가."
}
```

$combined = (0.70 \times 0.521 \times 0.880)^{(1/3)} = 0.685$. 기하평균이라 셋 중 하나만 낮아도 전체가 낮아진다.

`confidence.situation` 이 따로 필요한 이유 — `intrinsic` · `empirical` 은 둘 다 정책에 대한 확신이라 상황을 잘못 읽었을 가능성이 어디에도 반영되지 않는다. 오판 + 정책 자신감 = 높은 `combined` = 에스컬레이션 안 됨. `normal` 인지 정확도가 50.5%라 오탐이 구조적으로 많다 (`rationale/environment.md`).

record_escalation(...)

이 도구를 호출한 것 자체가 개입 1회다. 사람의 응답을 기다리지 않는다. 한 호출이 `kind: "escalation"` 레코드와 `kind: "decision"` 레코드를 같은 `step` 으로 둘 다 남긴다.

파라미터	타입	필수
<code>step</code>	int	예
<code>observation</code>	<code>Observation</code>	예
<code>situation</code>	<code>Situation</code>	예
<code>reason</code>	string	예
<code>confidence</code>	object	예 (<code>situation</code> 포함)

출력 필드	타입	설명
<code>escalation_id</code>	string	개입 1건의 식별자
<code>decision_id</code>	string	폴백 결정의 ID. <code>report_outcome</code> 에 쓴다
<code>fallback_policy</code>	<code>PolicyName</code>	항상 <code>"rule_based"</code>
<code>fallback_situation</code>	<code>Situation</code>	항상 <code>"normal"</code>
<code>fallback_allocation</code>	<code>SliceTriple</code>	폴백 정책이 낸 배분. <code>apply_allocation</code> 에 넣는다
<code>instruction</code>	string	에이전트에게 주는 다음 행동 지시

```
{"escalation_id": "exp-proposed-mixed-s0-esc-0031",  
  "decision_id": "exp-proposed-mixed-s0-0031",  
  "fallback_policy": "rule_based",  
  "fallback_situation": "normal",  
  "fallback_allocation": {"embb": 0.40, "urllc": 0.40, "mmtc": 0.20},  
  "instruction": "사람 호출을 기록했다. 대기하지 말고 fallback_allocation을 apply_allocation에 넣어 이  
  번 스텝을 진행한 뒤, step() 후 decision_id로 report_outcome을 호출하라."}
```

get_decisions(n, kind)

파라미터	타입	필수	기본	설명
n	int	아니오	50	최근 n건
kind	string	아니오	null	decision · escalation 필터

출력: [Decision] — decisions.json 의 레코드 그대로. ⑤가 채점했으면 outcome 이 붙어 있다.

```
[{"decision_id": "...-0012", "step": 12, "kind": "decision",  
  "situation": "emergency", "chosen_policy": "rule_based",  
  "allocation": {"embb": 0.20, "urllc": 0.70, "mmtc": 0.10},  
  "confidence": {"situation": 0.70, "intrinsic": 0.521, "empirical": 0.880, "combined": 0.685},  
  "outcome": {"sla_met": false, "error": 0.184, "scored_at_step": 13}}]
```

컨텍스트 주의 — 각 레코드에 observation 이 통째로 들어 있다. 에이전트용 호출은 $n \leq 10$. 전량 분석은 eval/score.py 가 파일을 직접 읽는다.

get_metrics(window)

파라미터	타입	필수	기본	설명
window	int	아니오	null	최근 n스텝만. null 이면 전체

출력: Metrics — 관측만으로 계산되는 지표뿐이다.

필드	타입	정의
<code>steps</code>	int	채점 완료된 결정 수 (= 기록 수 - 1)
<code>interventions</code>	int	<code>record_escalation</code> 호출 횟수
<code>autonomous_rate</code>	float	<code>1 - interventions / steps</code>
<code>sla_violations</code>	int	<code>outcome.sla_met == false</code> 인 결정 수. <code>observation.violations</code> 로 세지 않는다
<code>mean_utilization</code>	<code>SliceTriple</code>	평균 이용률
<code>policy_usage</code>	<code>{PolicyName: int}</code>	정책별 채택 횟수
<code>mttr</code>	float	위반 시작 스텝부터 전부 해소된 스텝까지의 평균 길이
<code>unresolved</code>	int	복구되지 않은 채 끝난 구간 수
<code>procurements</code>	int	<code>procure()</code> 호출 횟수
<code>procurement_cost_total</code>	float	조달 비용 합계
<code>mean_capacity</code>	<code>SliceTriple</code>	평균 용량 배수
<code>pressure_exceeded</code>	int	<code>demand_pressure ≥ 1.0</code> 이던 스텝 수

반환 금지 ⚠️: `perception_accuracy` · `escalation_precision`. 정답이 필요한 지표는 `eval/score.py` 가 오프라인으로 계산한다. (`flow/forbidden.md`)

집계 불변식

<code>steps</code>	= 채점된 결정 수	(60스텝 에피소드 → 59)
<code>sum(policy_usage)</code>	= 기록된 결정 수	(60)
$\sum \textcircled{2}$ 의 <code>n</code>	= 채점된 결정 수	(59)

`sum(policy_usage)` 와 $\sum n$ 이 1 차이나는 것은 정상이다. 2 이상 벌어지면 ②의 조용한 폴백이나 `report_outcome` 누락을 의심한다.

```
{
  "steps": 59,
  "interventions": 4,
  "autonomous_rate": 0.932,
  "sla_violations": 17,
  "mean_utilization": {
    "embb": 1.021,
    "urllc": 1.183,
    "mmtc": 0.742
  },
  "policy_usage": {
    "rule_based": 42,
    "lstm_forecast": 18,
    "dqn": 0
  },
  "mttr": 3.4,
  "unresolved": 1,
  "procurements": 3,
  "procurement_cost_total": 1875.0,
  "mean_capacity": {
    "embb": 1.600,
    "urllc": 1.783,
    "mmtc": 1.600
  },
  "pressure_exceeded": 21
}
```

⑤ `slice-feedback` — 계약

담당 C · `claude/spec/feedback.md`

결정을 채점하고 정책별 경험적 신뢰도를 유지한다. 설계 근거는 `rationale/feedback.md`.

`report_outcome(decision_id, observed)`

파라미터	타입	필수	출처
<code>decision_id</code>	string	예	④가 발급한 것
<code>observed</code>	Observation	예	<code>step()</code> 이후의 관측 (= <code>obs_{t+1}</code>)

반드시 `step()` 이후에 호출한다. `obs_{t+1}.allocation` 이 t에 적용된 배분, `obs_{t+1}.traffic` 이 그 배분이 감당해야 했던 수요다. `step()` 전 호출은 `premature_scoring` 오류 (`flow/errors.md`).

출력: Outcome

필드	타입	설명
<code>sla_met</code>	bool	<code>violations</code> 가 전부 <code>false</code>
<code>policy</code>	PolicyName	채점 대상 정책
<code>error</code>	float	0~1. 이상 배분과의 거리
<code>ideal_allocation</code>	SliceTriple	위반이 딱 없어지는 배분
<code>applied_allocation</code>	SliceTriple	실제 적용된 배분 (= <code>observed.allocation</code>)
<code>requested_allocation</code>	SliceTriple	에이전트가 요청했던 배분 (<code>decisions.json</code> 에서 조회)
<code>actuator_delta</code>	float	둘 사이의 거리. 액추에이터가 얼마나 꺾었나
<code>observed_violations</code>	SliceFlags	채점 시점의 위반. MTTR 재구성용
<code>vendor_id</code>	string null	있으면 에이전트가 ③ <code>update_rating</code> 에 증계
<code>reliability_before</code> · <code>reliability_after</code>	float	

error 계산 — 정답을 쓰지 않고 관측만으로 나온다. `capacity` 를 빼면 안 된다.

```
a*    = normalize(traffic / (thresholds × capacity))  # 이상 배분
error = L1(applied_allocation, a*) / 2                # [0, 1]
```

신뢰도 갱신: `r ← (1-0.2)·r + 0.2·(sla_met ? 1 : 0)`

부작용: ④의 해당 레코드에 `outcome` 을 덧붙인다 (`sla_met`, `error`, `scored_at_step`, `applied_allocation`, `requested_allocation`, `actuator_delta`, `observed_violations`).

```
// 요청 - step() 후의 관측
{"decision_id": "exp-proposed-emergency-s0-0012",
 "observed": {"step": 13,
  "traffic": {"embb": 0.571, "urllc": 0.595, "mmtc": 0.390},
  "allocation": {"embb": 0.340, "urllc": 0.490, "mmtc": 0.170},
  "capacity": {"embb": 1.600, "urllc": 1.600, "mmtc": 1.600},
  "utilization": {"embb": 1.050, "urllc": 0.760, "mmtc": 1.435},
  "violations": {"embb": true, "urllc": false, "mmtc": true}, ...}}
// 응답
{"sla_met": false,
 "policy": "rule_based",
 "error": 0.184,
 "ideal_allocation": {"embb": 0.392, "urllc": 0.306, "mmtc": 0.301},
 "applied_allocation": {"embb": 0.340, "urllc": 0.490, "mmtc": 0.170},
 "requested_allocation": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
 "actuator_delta": 0.210,
 "observed_violations": {"embb": true, "urllc": false, "mmtc": true},
 "vendor_id": null,
 "reliability_before": 0.880,
 "reliability_after": 0.704}
```

계산 확인: $a^* = \text{normalize}([0.571/(0.9 \times 1.6), 0.595/(1.2 \times 1.6), 0.390/(0.8 \times 1.6)]) = [0.392, 0.306, 0.301]$, $\text{error} = (|0.340 - 0.392| + |0.490 - 0.306| + |0.170 - 0.301|) / 2 = 0.184$, $r = 0.8 \times 0.880 = 0.704$.

이 예시가 `rule_based` 의 한계를 보여준다. `situation="emergency"` 의 고정 목표 `[0.2, 0.7, 0.1]` 은 과잉 반응이었다 — URLLC에 0.49를 줬는데 이용률 0.76으로 여유가 남았고, 굵긴 mMTC가 1.435로 터졌다. 상황 판단은 맞았는데 정책이 틀렸다. ⑤가 `rule_based` 의 신뢰도를 낮춘다.

get_reliability_table()

입력: 없음 · 출력: `ReliabilityTable`

필드	타입	설명
<code>{policy}.reliability</code>	float	EMA 원값
<code>{policy}.n</code>	int	누적 표본 수
<code>{policy}.effective</code>	float	축소 보정값. 에이전트는 이걸 쓴다 → ④ <code>confidence.empirical</code>
<code>{policy}.recent_error</code>	float	최근 5회 <code>error</code> 의 EMA. ② <code>propose_allocation.recent_error</code> 에 중계

축소 보정: $\text{effective} = (r \cdot n + 0.5 \cdot 5) / (n + 5)$

```
{"rule_based": {"reliability": 0.910, "n": 120, "effective": 0.894, "recent_error": 0.094},
"lstm_forecast": {"reliability": 0.840, "n": 95, "effective": 0.823, "recent_error": 0.112},
"dqn": {"reliability": 0.420, "n": 6, "effective": 0.456, "recent_error": 0.380}}
```

두 종류의 신뢰도 — 종합은 에이전트만 한다.

	제공	의미
내재적 (<code>confidence</code>)	②	이번 입력에 대한 확신
경험적 (<code>effective</code>)	⑤	이 정책이 평소 얼마나 맞았나

```
combined = (situation × intrinsic × effective)^(1/3)
escalate if combined < 0.45
```

셋의 기하평균이다. `situation` 은 에이전트가 내는 상황 판단 자체에 대한 확신이며 ②·⑤ 어느 쪽도 주지 않는다. 이게 없으면 상황을 잘못 읽고도 정책에 자신 있을 때 `combined` 가 높게 나와 에스컬레이션이 일어나지 않는다. `normal` 인지 정확도가 50.5%라 오탐이 구조적으로 많다 (`rationale/environment.md`).

계약 검증 — 값 유실 20건

담당 C · claude/rationale/verification.md

생산자 없는 소비 · 소비자 없는 생산 · 근본 원인 3패턴 · 재발 방지

1.5 도구 계약 검증 — 값 유실 9건

정정 A~K가 기존 코드의 문제였다면, 이 절은 **TOOLS.md**의 도구 계약 자체를 검증한 결과다. 도구 21개의 모든 입력 필드에 대해 "이 값을 누가 생산하는가"를, 모든 출력 필드에 대해 "이 값을 누가 소비하는가"를 역추적했다.

생산자 없는 소비 4건, 소비자 없는 생산 5건이 나왔다. 전부 코드 버그가 아니라 계약의 구멍이며, 구현 전에 메우지 않으면 통합 시점에 드러난다.

#	값	증상	등급	담당
V2	decision_id (에스컬레이션 시)	그 스텝의 SLA 결과가 통계에서 소멸	치명	A
V3	run_id	truth.jsonl × decisions.json 조인 불가	치명	A · C
V4	recent_error	②의 LSTM 신뢰도를 계산할 수 없음	중대	B · C
V5	applied_allocation	기록(요청)과 채점(적용)의 기준 불일치	중대	B
V6	outcome.sla_met	sla_violations 가 1스텝 어긋남	중대	A
V7	in_distribution	논문 주장을 하되 검증 불가	경미	A · B
V8	demand_class	분류기 오류와 판단 오류를 구분 불가	경미	A · C
V9	situation ↔ truth 매핑	채점 기준이 사후 해석 가능	경미	A
V10	$\sum$ policy_usage vs $\sum$ n	정상 차이를 버그로 오인	경미	A

도구별 반영 내용은 **TOOLS.md**의 해당 절에 ⚠️ 블록으로 들어가 있다. 여기서는 왜 생겼는가와 어떻게 막는가만 적는다.

근본 원인 — 세 가지 패턴

아홉 건이 무작위로 생긴 게 아니다. 서버를 나눈 방식에서 구조적으로 유도된다.

패턴 1 — 호출 순서가 값의 가용 시점을 어긋나게 한다 (V5, V6)

record_decision (3단계)이 apply_allocation (4단계)보다 먼저다. 기록 시점에 "실제 적용된 값"은 아직 존재하지 않는다. 그런데 채점은 적용값 기준이다.

기록 시점 **t**: 요청 `allocation = {0.20, 0.70, 0.10}` ← ④에 남는 것
적용 시점 **t**: 실제 `allocation = {0.34, 0.49, 0.17}` ← 아무데도 안 남던 것
채점 시점 **t+1**: `error`는 적용값 기준 = `0.086`
요청값 기준이었다면 = `0.316` ← 4배 차이

같은 이유로 ④가 보는 `observation.violations` 는 직전 결정의 결과라 1스텝 어긋난다.

판정법: 도구 X의 출력이 도구 Y의 입력인데 호출 순서가 $Y \rightarrow X$ 라면, 그 값은 유실된다. 순서를 바꾸거나, 나중에 그 값을 보는 제3자(여기서는 ⑤)가 채워 넣어야 한다.

패턴 2 — 무상태 원칙이 누적값의 공급선을 끊는다 (V4)

②는 무상태이고 ⑤를 호출할 수 없다(서버 간 직접 호출 금지, §2.2). 그런데 `lstm_forecast` 의 신뢰도 `exp(-3 \cdot \bar{e})` 에서 $\bar{e}$ 는 누적 오차다. ②가 원리적으로 알 수 없는 값을 식에 넣어 두었다.

해결은 §2.2의 원칙 그대로다 — 에이전트가 중계한다. ⑤가 `recent_error` 를 반환하고, 에이전트가 `propose_allocation` 의 인자로 넘긴다. ②는 여전히 무상태다.

판정법: 무상태 서버의 산출식에 "최근 N회", "누적", "평균" 같은 말이 들어가면 그 값은 반드시 인자여야 한다.

패턴 3 — 식별자를 쓰기만 하고 발급하지 않는다 (V2, V3)

`decision_id = {run_id}-{step:04d}` 라고 형식만 정하고 `run_id` 를 발급하는 도구를 두지 않았다. 에스컬레이션도 마찬가지로 `escalation_id` 만 주고 `decision_id` 를 안 줘서, 그 스텝은 `report_outcome` 을 호출할 방법이 없었다.

판정법: 명세에 등장하는 모든 식별자에 대해 "이걸 처음 만드는 도구가 무엇인가" 를 답할 수 있어야 한다. 답이 없으면 유실이다.

V2가 가장 위험한 이유

나머지 여덟 건은 지표가 틀리거나 분석이 제약되는 수준이다. V2만 논증 자체를 무너뜨린다.

에스컬레이션한 스텝은 정의상 에이전트가 가장 자신 없던 스텝이다. `decision_id` 가 없어 채점되지 않으면 그 스텝들이 통계에서 통째로 빠진다.

proposed arm: 어려운 스텝 4개를 에스컬레이션 → 그 4개가 집계에서 제외
baseline arm: 에스컬레이션 없음 → 60개 전부 집계

비교군마다 표본이 다르다. `proposed` 의 `sla_violations` 가 낮게 나와도 그건 어려운 스텝을 빼고 센 결과다. 핵심 논증인 "개입은 줄었는데 SLA 위반은 늘지 않았다" 가 편향된 표본 위에서 성립한다. 심사에서 지적당하면 방어할 말이 없다.

→ `record_escalation` 이 `escalation_id` 와 `decision_id` 를 한 번의 호출로 둘 다 발급한다. 나눠 두면 에이전트가 빠뜨릴 수 있다.

재발 방지 — 계약 검사

도구는 앞으로도 늘어난다(이미 20 → 21). 사람이 매번 역추적하면 또 놓친다.

`docs/contract.json`에 기계가 읽을 수 있는 계약을 둔다. 도구마다 필수 입력, 출력 필드, 그리고 각 필수 입력의 생산자를 명시한다.

```
{
  "propose_allocation": {
    "server": "policy",
    "requires": {
      "observation": "observe.get_observation",
      "situation": "AGENT",
      "recent_error": "feedback.get_reliability_table"
    },
    "produces": ["policy", "allocation", "confidence", "in_distribution", "status"]
  }
}
```

`tools/check_contract.py`가 검사할 것:

1. 모든 필수 입력에 생산자가 있는가 (AGENT 명시 포함) → V3, V4 유형 차단
2. 생산자의 호출 순서가 소비자보다 앞인가 → V5, V6 유형 차단
3. 명세에 등장하는 모든 식별자에 발급 도구가 있는가 → V2 유형 차단
4. `produces` 중 아무도 소비하지 않는 필드 목록 → V7, V8 유형 경고 (의도적일 수 있으므로 경고)

AGENT가 생산자인 값은 네 개뿐이고, 그 넷이 연구의 기여 지점 전체다 — `situation`, `confidence.combined`, 에스컬레이션 판단, 정책 선택. 이 목록이 늘어나면 에이전트가 해야 할 판단이 서버로 새고 있다는 뜻이므로, 검사기가 넷을 넘으면 경고하도록 한다.

소유는 C다. 목 서버와 같은 성격 — 계약의 실행 가능한 명세이고, C는 A-B의 서버를 소비하는 쪽이라 불일치를 가장 먼저 만난다.

1.6 전면 재검증 — 추가 11건

§1.5 이후 `capacity` 도입(정정 K)으로 계약이 바뀌었다. 세 문서를 교차 대조하고 모든 예시를 재계산한 결과 11건이 더 나왔다. 절반은 `capacity` 도입이 만든 파급이고, 절반은 처음부터 있던 사각지대다.

#	항목	증상	등급	담당
W1	조달이 루프의 어디에 들어가는가	순서 미정. <code>add_capacity</code> 가 <code>step()</code> 뒤로 가면 조달한 스텝이 효과를 못 봄	중대	C
W2	③이 현재 스텝을 모른다	<code>expires_at_step</code> 을 계산할 수 없음	중대	B
W3	<code>cost_total</code> 단위 불일치	시간당 단가 × 스텝 수 → 비용 4배 부풀림	중대	B
W4	⑤가 <code>vendor_id</code> 를 모른다	<code>update_rating</code> 중계 불가	중대	A · B
W5	실행 생명주기 미정	60회 산출물이 한 파일에 덮어쓰기	치명	A · C
W6	<code>compare_policies</code> 의 <code>recent_error</code>	스칼라라 전 정책이 같은 오차 공유	중대	B
W7	① 도구 목록에 <code>add_capacity</code> 누락	설계서 ↔ 명세서 불일치	경미	—
W8	LSTM이 <code>capacity</code> 를 못 봄	11차원 고정. 한계로 명시 필요	경미	B
W9	<code>demand_pressure</code> 계산 오류	1.061 → 1.069	경미	—
W10	컨텍스트 예산이 20개 기준	21개 + <code>Observation</code> 확장 반영	경미	C
W11	오류 규약에 조달 케이스 없음	상한 초과·순서 위반 미정의	경미	A · B

W5가 가장 위험하다

§1.5의 V3(`run_id` 부재)를 고치고도 절반만 해결했다. `run_id` 를 발급하게 만들었지만, 산출물 경로는 여전히 `data/decisions.json` 단일이었다.

```
exp-baseline-normal-s0    → data/decisions.json (덮어씀)
exp-arm1-normal-s0       → data/decisions.json (덮어씀)
...60회...
                          → 마지막 1회분만 남음
```

`reset()` 이 ①만 초기화한다는 점도 명시되지 않았다. ③의 벤더 레이팅과 ⑤의 신뢰도는 실행을 넘어 살아남는데, 그게 의도인지 버그인지 문서 어디에도 없었다. `cold / warm` 모드는 §6.2에 있었으나 파일 경로와 연결되지 않아 실행 불가능한 규약이었다.

→ §5.0에 실행 생명주기를 명시했다. `runs/{run_id}/` 로 분리하고, 서버별로 무엇이 초기화되고 무엇이 남는지 표로 고정했다.

W1·W2·W3 — capacity 도입의 파급

정정 K는 ①에 용량을 넣는 것으로 끝나지 않았다. 조달이 실제로 동작하게 되자 그동안 무해했던 공백이 드러났다.

- **W1:** 조달이 장식일 때는 `procure()` 를 언제 부르든 상관없었다. 이제 `add_capacity` 가 `step()` 뒤로 가면 조달한 스텝이 효과를 못 본다 — 에이전트 입장에서 “샀는데 아무 일도 안 일어났다”가 되어 조달 학습이 왜곡된다.
- **W2:** 만료가 없을 때는 `expires_at_step` 이 의미 없었다. 이제 ③이 그 값을 계산해야 하는데 ①과 별개 프로세스라 현재 스텝을 모른다. 에이전트가 `current_step` 을 넘겨야 한다.
- **W3:** 비용이 집계되지 않을 때는 단위가 무의미했다. 이제 `procurement_cost_total` 이 논문 지표이므로 시간당 단가 × 스텝 수를 그대로 곱하면 4배 부풀려진다.

```
cost_total = cost × duration_steps × (MINUTES_PER_STEP / 60)
            = 250 × 10 × 0.25 = 625.0          # 2500.0 이 아니다
```

교훈: 죽은 기능은 계약의 구멍을 감춘다. 정정 K로 ③이 살아나자 W1·W2·W3가 동시에 드러났다. 1차 범위에서 제외한 `dqn` 도 2차에 넣는 순간 같은 일이 일어날 것이다.

W6 — V4를 고치면서 만든 새 구멍

§1.5의 V4를 고쳐 `propose_allocation` 에 `recent_error` (스칼라)를 넣었다. 그런데 `compare_policies` 는 정책을 동시에 평가한다. 스칼라를 받으면 모든 정책이 같은 오차 이력을 공유하는 셈이라 신뢰도 비교가 무의미해진다.

→ `compare_policies` 는 `recent_errors: {policy: float}` 딕셔너리를 받는다. ⑤의 `get_reliability_table()` 출력을 그대로 넘기면 된다.

수정이 새 구멍을 만든다. W6은 V4의 직접적 부산물이다. §1.5의 계약 검사기(`check_contract.py`)를 수정할 때마다 돌려야 하는 이유다.

환경 검증 — 실험에 신호가 있는가

담당 C · claude/rationale/environment.md

시나리오별 압력 실측 · 상수 재조정 · 상황 인지 상한

1.7 환경 검증 — 실험에 신호가 있는가

계약이 맞아도 환경이 잘못 조율되어 있으면 결과가 안 나온다. 시나리오별로 트래픽 생성기를 직접 돌려(60스텝 × 시드 10~20) 확인했다.

발견 1 — 평시에도 절반 이상이 재배분으로 해결 불가였다

시나리오	평균 demand_pressure	1.0 초과 비율
normal	1.049	53%
emergency	1.204	74%
special_event	1.237	78%
iot_surge	1.189	70%

demand_pressure > 1.0 은 "어떤 배분으로도 전 슬라이스 SLA를 지킬 수 없다"를 뜻한다(§ TOOLS.md 2.1).
평시의 절반이 그 상태라면:

- SLA 위반 지표가 포화되어 비교군 간 차이가 드러나지 않는다
- demand_pressure ≥ 1.0 을 조달 트리거로 쓰면 에이전트가 평시에도 계속 산다

원인은 시간 요인이다. daily_factor (±0.3)와 weekly_factor (+0.2)를 합하면 평시 피크가 1.5배인데, 이벤트 효과(URLLC 2배 = 전체 압력 +26%)와 비슷하거나 더 크다. START_HOUR = 8 이 하필 고부하 구간이었다.

조정 — 상수 세 개

원 코드의 상수(THRESHOLDS , BASE_TRAFFIC , 시간 계수)는 건드리지 않는다. 조정 대상은 이 설계가 새로 도입한 값뿐이다.

상수	기존	변경	근거
CAPACITY_BASE	(없음, 1.0)	1.6	평시 여유 확보. 정정 K가 도입한 개념이므로 기준점 선택이 자유롭다
CAPACITY_MAX	2.0	2.6	기본 + 조달 4회분
START_HOUR	8	0	자정 시작. 60스텝(15시간)이 저부하→피크를 모두 덮는다

조정 후 실측:

시나리오	<code>demand_pressure</code> > 1.0	초기 배분에서 위반 스텝
<code>normal</code>	5.6%	68.4%
<code>emergency</code>	27.7%	75.2%
<code>special_event</code>	34.3%	94.2%
<code>iot_surge</code>	28.5%	91.7%

두 숫자의 관계가 실험의 구조다.

- 위반은 어디서나 흔하다 (68~94%) → 정책이 할 일이 있다. 초기 배분 `[0.4, 0.4, 0.2]` 은 어느 시나리오에도 최적이지 아니다.
- 평시의 위반은 94%가 재배분으로 고칠 수 있다 → 조달 없이 해결하는 것이 정답이다.
- 이벤트에서는 28~34%가 재배분으로 불가 → 여기서만 조달이 정답이 된다.

즉 "고칠 수 있는가"의 분포가 시나리오를 가른다. 이게 에이전트가 재배분과 조달을 구분해 배워야 할 신호다.

발견 2 — 조달로도 다 해결되면 안 된다

압력이 1.0을 넘는 비상 스텝을 1.0 아래로 내리는 데 필요한 조달 횟수: 평균 2.1회, 중앙값 2회, 최대 8회 (`GAIN_SCALE` = 0.25 기준).

`CAPACITY_MAX` 를 3.6 (= 1.6 + 8×0.25)으로 잡으면 모든 스텝이 조달로 해소 가능해진다. 그러면 어려운 상황이 사라져 에스컬레이션이 무의미해진다. 2.6 에서 막으면 중앙값 사례는 해결되고 극단 사례는 남는다.

발견 3 — 상황 인지의 상한은 약 72%

절대량이 아니라 구성비가 신호다. 구성비만으로 최근접중심 분류 시 71.7%, `normal` 만 50.5% (무작위 25%). 상세는 `TOOLS.md` §10.

이는 나쁜 소식이 아니다. 천장이 100%가 아니라 비교군 간 차이가 드러날 여지가 있다는 뜻이다. 다만 `normal` 50.5%는 오탐이 구조적으로 많다는 뜻이므로, 상황 판단 자체의 확신(`confidence.situation`)을 에스컬레이션 경로에 넣어야 한다 — `TOOLS.md` §5.1.

컨텍스트 예산 (에이전트 런타임)

담당 C · `claude/rationale/context-budget.md`

도구 설명과 반환값이 매 턴 LLM 컨텍스트를 차지한다. 21개 도구의 설명만으로 이미 상당하다.

항목	대략
도구 21개의 이름 + 설명 + 스키마	2,600 ~ 3,700 토큰 (상주)
<code>Observation</code> 1건	약 210 토큰 (<code>capacity</code> · <code>demand_pressure</code> 추가분 포함)
<code>PolicyProposal</code> 1건	약 120 토큰
<code>compare_policies</code> (3정책)	약 360 토큰
<code>score_offerings</code> (벤더 5)	약 250 토큰
<code>get_decisions(n=50)</code>	약 10,500 토큰 ⚠
<code>explain_score</code> (<code>neural_network</code> 포함 시)	약 500 토큰 (제거하면 200)

주의할 호출 세 개

- `get_decisions(n=50)` — 각 레코드에 `observation` 이 통째로 들어 있고, V7·V8로 필드가 늘었다. 에이전트용은 `n ≤ 10`.
- `compare_policies` — `propose_allocation` 의 3배. 정책 전환을 고민할 때만.
- `record_decision` — 입력이 크다. `observation` + `considered` + `demand_class` 를 매 스텝 보낸다. 출력은 `decision_id` 하나뿐이지만 요청 토큰이 관측 1건 이상이다.

규모

스텝당 3~5회 × 120스텝 × 60회 실행 ≈ 2~3만 호출. 모델 선정 전에 예산을 산정해야 한다 (`MCP-design.md` §9-5, 미해소).

산출식

담당 C · claude/build/formulas.md

내재적 신뢰도 · 경험적 신뢰도 · 도구 설명 수위

6. 미결 사항 해소안

6.1 내재적 신뢰도 산출식 (§9 3번)

정책	산출식	근거
rule_based	$0.50 + 0.30 \cdot \min(1, m_{in})$ $m_{in} = \min_i \setminus u_i - \theta_i \setminus / \theta_i$	이용률이 임계값에 바짝 붙어 있으면 규칙의 이산 분기가 자의적 → 확신 낮음. 여유가 크면 분기가 명확. 범위 [0.50, 0.80]
lstm_forecast	$\exp(-3 \cdot \bar{e}) \cdot 1[in_distribution]$ $\bar{e} = \text{최근 5회 error의 EMA}$ (§4 ⑤ 정의)	모델에 추론 시 dropout이 없어 분산 추정이 불가. 최근 실적이 유일하게 값싼 대리 지표. $\bar{e}=0.1 \rightarrow 0.74$, $\bar{e}=0.3 \rightarrow 0.41$
dqn	$\sigma(Q_{(1)} - Q_{(2)}) \cdot 1[in_distribution]$	상위 2개 Q값 마진. 2차

rule_based의 하한을 0.50으로 잡은 것은 의도적이다. **항상 가용한 안전 기본값**이므로 다른 정책이 전부 실패해도 이것 하나는 에스컬레이션 문턱($\tau=0.45$) 위에 있다. 이 상수가 곧 "언제 사람을 부를 것인가"의 하한을 정의한다.

첫 5스텝은 $\bar{e}$ 표본이 없다. lstm_forecast는 status="unavailable" (history < 10)이므로 자연히 배제된다 — 별도 처리 불필요.

6.2 경험적 신뢰도 갱신식 (§9 4번)

```
r ← (1 - α)·r + α·s      α = 0.2,  s = 1 if sla_met else 0
n ← n + 1
```

```
effective = (r·n + r₀·m) / (n + m)      r₀ = 0.5,  m = 5
```

- $\alpha = 0.2$ — 반감기 약 3스텝. 시나리오가 전환되는 mixed (120스텝, 5회 전환)에서 전환 후 5~10스텝 내에 신뢰도가 따라붙는다. α 가 작으면 mixed에서 자기 개선이 안 보이고, 크면 잡음을 학습한다.
- **축소 항 (m=5)** — n이 작을 때 r이 0 또는 1로 튀는 것을 막는다. 첫 성공 1회로 신뢰도 1.0이 되면 에이전트가 그 정책에 고착된다.
- **에이전트의 종합** (분리 설계서 §2.2):

```
combined = sqrt( intrinsic × effective )      # 기하평균
escalate if combined < τ,    τ = 0.45         # 사전 확정
```

기하평균을 쓰는 이유: 한쪽이 낮으면 전체가 낮아진다. 산술평균은 "이번엔 확신 없지만 평소 잘 맞으니 간다"를 허용하는데, 그건 에스컬레이션이 필요한 바로 그 상황이다.

$\tau = 0.45$ 고정. 실험 전에 확정하고 절대 조정하지 않는다. 조정하면 개입 횟수를 원하는 대로 만들 수 있어 결과가 무의미해진다. τ 민감도는 **사후 분석**으로 별도 보고한다 (`decisions.json` 에 `combined` 를 기록해 두었으므로 재실행 없이 τ 를 바꿔 곡선을 그릴 수 있다).

에피소드 간 신뢰도 유지 여부 — 두 모드를 모두 지원하고 실험 변수로 삼는다.

모드	<code>reliability.json</code>	보이는 것
<code>cold</code>	시나리오마다 초기화	단일 에피소드 내 적응
<code>warm</code>	전체 실행에 걸쳐 누적	장기 자기 개선 (논문의 §5 주장)

주 결과는 `warm`. `cold` 는 대조군.

6.3 도구 설명 수위 (§9 8번)

분리 설계서의 지적이 정확하다. MCP는 도구 설명을 LLM 컨텍스트에 그대로 넣으므로, "평시 효율 우수" 는 사실상 프롬프트다.

두 벌을 `mcp/policy/descriptions.py` 에 두고 환경변수로 전환한다.

```
DESC = {
  "minimal": {
    "rule_based": "임계값 기반 배분. 상황 라벨을 입력으로 받는다.",
    "lstm_forecast": "시계열 모델 기반 배분. 관측 이력 10스텝 필요.",
    "dqn": "강화학습 정책 기반 배분.",
  },
  "advisory": {
    "rule_based": "이율을 임계값 기반. 항상 가용, 보수적.",
    "lstm_forecast": "시계열 예측 기반. 평시 효율 우수.",
    "dqn": "강화학습 정책. 학습 분포 내에서 우수.",
  },
}
MODE = os.environ.get("SLICE_DESC_MODE", "minimal")
```

`minimal` 판은 무엇을 하는지와 전제조건만 말하고 언제 쓰라는 말을 하지 않는다. `lstm_forecast` 의 "이력 10스텝 필요" 는 조언이 아니라 호출 규약이므로 남긴다.

주 실험은 `minimal` 로 돌린다. `advisory` 는 비교 실행으로 붙인다.

- 두 조건의 정책 선택 분포가 비슷 → 에이전트가 관측으로 판단했다는 증거
- 크게 다름 → 설명에 이끌렸다는 증거이며, 이것도 **보고할 가치가 있는 결과**다 (MCP 기반 에이전트 설계의 일반 교훈)

어느 쪽이든 §9 8번이 위험에서 기여로 바뀐다. `desc_mode` 를 `decisions.json` 의 `config` 에 기록한다.

구현 순서와 미결 사항

담당 C · claude/build/order.md

단계별 완료 판정 · ③ 서버 골격 · 남은 결정

8. 구현 순서와 체크포인트

분리 설계서 §7을 유지하되, 각 단계의 완료 판정 기준을 붙인다.

순서	대상	완료 판정	작업량
0	모델 적재 검증	§7의 6개 항목 통과. <code>ranges.json</code> 생성	小
1	③ <code>slice-market</code>	에이전트가 <code>score_offerings</code> 호출 → 5개 벤더 점수 수신. MCP 배선 검증	小
2	① <code>slice-observe</code>	동일 시드 2회 실행 → <code>truth.jsonl</code> 바이트 단위 동일 (정정 F 검증)	中
3	④ <code>slice-audit</code>	<code>get_metrics()</code> 반환 필드에 정답 의존 지표 부재 확인 (정정 D)	小~中
4	② <code>slice-policy</code>	<code>situation</code> 인자로 배분이 바뀜(정정 E). LSTM 실패 시 <code>status="unavailable"</code> 반환(정정 H)	中
5	⑤ <code>slice-feedback</code>	<code>warm</code> 모드 120스텝에서 <code>reliability.json</code> 의 값이 실제로 움직임	中
6	<code>eval/score.py</code>	<code>truth.jsonl</code> × <code>decisions.json</code> 조인 → <code>perception_accuracy</code> 산출	小
7	베이스라인 실행	비교군 4개 × 시나리오 5개 × 시드 3개 = 60회	中
8	② <code>dqn</code> 추가	선택. 데이터 재생성 선행	大

2단계의 완료 판정이 가장 중요하다. 동일 시드 재현이 안 되면 이후 모든 비교가 무의미하다. 여기서 반드시 멈추고 확인한다.

1단계(③)를 먼저 하는 이유는 분리 설계서와 같다 — **MCP 배선 자체를 검증**하는 것이 목적이지 ③이 중요해서가 아니다. FastMCP 서버 하나, stdio 연결 하나, 도구 호출 하나가 왕복하면 나머지는 같은 패턴의 반복이다.

③ 서버 골격 (1단계 산출물 예시)

```
# mcp/market/server.py
from fastmcp import FastMCP
from mcp.common.store import read_json, write_json_atomic
from mcp.common.paths import VENDORS
from mcp.market.scoring import score_vendor_offering, get_score_breakdown

mcp = FastMCP("slice-market")

@mcp.tool()
def score_offerings(slice_type: str, qos_requirements: dict) -> list[dict]:
    """주어진 slice_type과 QoS 요구에 대해 등록된 모든 벤더를 점수화한다."""
    vendors = read_json(VENDORS)
    out = [{"vendor_id": v["id"], "name": v["name"],
            "score": score_vendor_offering(v, slice_type, qos_requirements),
            "rating": v["rating"]}
            for v in vendors if slice_type in v.get("offerings", {})]
    return sorted(out, key=lambda x: x["score"], reverse=True)

@mcp.tool()
def update_rating(vendor_id: str, outcome: dict) -> dict:
    """결과 보고를 반영해 벤더 레이팅을 갱신한다. ㉔의 보고를 에이전트가 중계한다."""
    vendors = read_json(VENDORS)
    delta = 0.05 if outcome.get("sla_met") else -0.20
    for v in vendors:
        if v["id"] == vendor_id:
            v["rating"] = max(1.0, min(5.0, v["rating"] + delta))
            write_json_atomic(VENDORS, vendors)
            return {"vendor_id": vendor_id, "rating": v["rating"], "delta": delta}
    return {"error": f"unknown vendor: {vendor_id}"}

if __name__ == "__main__":
    mcp.run()
```

도구 `docstring`이 곧 LLM 프롬프트라는 점을 여기서부터 의식한다 (§6.3).

9. 남은 미결 사항

분리 설계서 §9 중 이 문서에서 해소되지 않은 것.

#	항목	상태
0	TF 적재 검증	\$7로 절차 확정. 실행 대기
1	DQN 1차 포함 여부	해소 — 포함하지 않음 (§7)
2	DQN 상태의 <code>is_emergency</code> 차원	2차로 이월. 에이전트 추론값 주입이 기본안
3	내재적 신뢰도 산출식	해소 (§6.1)
4	경험적 신뢰도 갱신식	해소 (§6.2)
5	오케스트레이터 LLM 선택·비용	미해소. 스텝당 3~5회 도구 호출 × 120스텝 × 60회 실행 ≈ 2~3만 호출. 모델 선정 전 토큰 예산 산정 필요
6	문헌 조사	미해소 (3단계)
7	Dashboard API 재활용	미해소 (6단계)
8	도구 설명 수위	해소 — 실험 변수로 승격 (§6.3)
9	에이전트 프롬프트 배경 지식 범위	미해소. §6.3과 같은 문제가 프롬프트에도 있음. <code>decisions.json</code> 에 프롬프트 해시를 기록해 사후 추적 가능하게 할 것
10	조달의 환경 반영 방식	해소 — ①에 용량 배수 도입 (정정 K)
11	조달 비용 억제 장치	해소 — 예산 대신 상한·만료 + 비용을 논문 지표로 보고

5번이 다음 병목이다. 2~3만 회의 도구 호출은 비용과 시간 모두에서 무시할 수 없다. 2단계 개발계획서에서 다음을 정해야 한다.

- 모델 선정 (로컬 Ollama vs API)
- 시드 3개가 맞는지, 시나리오 5개를 다 돌릴지
- 스텝당 호출 횟수 상한 (도구 호출 루프가 발산하지 않도록)

9번은 8번과 같은 위험이다. 도구 설명을 최소판으로 만들어도 시스템 프롬프트에 "URLLC 이용률이 높으면 비상 상황일 수 있다" 라고 적으면 같은 문제가 된다. 프롬프트도 최소판/조언판 두 벌을 준비하고 `config` 에 기록한다.

C · 에이전트 — 작업 목록

담당 C · claude/team/roles.md

4. C — 에이전트

한 줄 요약: 논문의 주인공. 코드량은 적고 반복 튜닝이 많다.

4.1 작업 목록

C-0. 목 서버 5개 (Day 1) ⚠️ 최우선

C는 ①~⑤가 다 있어야 루프를 돌릴 수 있습니다. 그대로 기다리면 마지막 주까지 아무것도 못 합니다.

mcp/mock/ 에 고정 응답만 돌려주는 5개 서버를 Day 1에 만드세요. mcp/common/schema.py 만 있으면 됩니다.

```
# mcp/mock/observe.py - 30줄이면 충분
@mcp.tool()
def get_observation() -> dict:
    return {"step": 12, "sim_time": {...},
            "traffic": {"embb": 0.52, "urllc": 0.61, "mmtc": 0.19},
            "utilization": {"embb": 1.30, "urllc": 1.52, "mmtc": 0.95},
            "violations": {"embb": True, "urllc": True, "mmtc": True}, ...}
```

A의 fixtures/ 가 나오면 목이 그걸 재생하게 바꿉니다. 그러면 실제 시나리오 전개에 대고 프롬프트를 다듬을 수 있습니다.

목 서버가 계약의 실행 가능한 명세가 됩니다. A·B가 진짜 서버를 완성한 뒤 목과 응답 형식이 다르면 그게 계약 위반입니다. 3인 병렬 개발에서 스키마 불일치를 잡는 가장 값싼 장치입니다.

C-1. 오케스트레이터 루프 — orchestrator/run.py

설계서 §3.1의 순서를 그대로 구현합니다. 순서가 곧 지표의 정의입니다.

```
1. obs_t = ①.get_observation()          # util_t 는 a_{t-1}의 성적표
2. [판단] ②.classify_demand / propose_allocation · ⑤.get_reliability_table
3. ④.record_decision(...) → decision_id # 판단을 먼저 기록 (실행 전)
4. ①.apply_allocation(a_t)
5. ①.step()
6. ⑤.report_outcome(decision_id, obs_{t+1})
```

규칙 셋: 1. 채점은 1스텝 지연됩니다. report_outcome 은 반드시 step() 이후에. 2. 에피소드 마지막 결정은 채점 불가. 집계에서 제외 (60스텝 → 유효 59). 3. 기록(3)이 실행(4)보다 먼저. 실행 후 기록이면 실패 사례가 사라져 자율 처리율이 과대평가됩니다.

루프는 파이썬이 돌립니다 — LLM에게 순서를 맡기지 마세요 ⚠

MCP는 도구를 줄 뿐 호출 순서를 강제하지 않습니다. 스텝당 6~10회 × 120스텝 × 60회 동안 LLM은 반드시 `report_outcome` 을 빠뜨리고, 순서를 뒤집고, 20~30스텝 뒤 드리프트합니다. 호출하지 않은 것은 오류를 내지 않으므로 대부분 조용히 누락됩니다.

```
for t in range(total_steps):
    obs = observe.get_observation()
    rel = feedback.get_reliability_table()

    j = ask_llm(obs, rel, recent)      # ← LLM은 여기서만
    # situation · confidence.situation · policy · procure? · escalate?

    if j.procure: ...                  # 2b
    audit.record_decision(...)         # 3   ← 파이썬이 보장
    observe.apply_allocation(...)      # 4
    observe.step()                    # 5
    feedback.report_outcome(...)       # 6
```

LLM이 답할 것은 네 가지뿐입니다. \$1.5의 “AGENT 가 생산자인 값은 네 개”와 정확히 일치합니다.

논문에도 유리합니다. 자율성 주장은 판단에 관한 것이지 도구 호출 순서가 아닙니다. 루프를 고정하면 비교군 간 차이가 전부 판단에서 나오고, “루프를 잘못 돌아서 SLA가 나뉘었다”는 반론이 차단됩니다.

컨텍스트는 스텝마다 재구성합니다

120스텝 대화를 누적하면 후반에 100k 토큰을 넘습니다. 대화 이력을 이어붙이지 마세요.

시스템 프롬프트 (고정) + 현재 관측 1건 + 신뢰도 표 + 최근 5스텝 요약(한 줄씩)

최근 이력은 원본이 아니라 요약입니다. `get_decisions(n=5)` 를 그대로 넣으면 관측이 팔려와 5배가 됩니다.

중계 책임 — 서버끼리 대화하지 않으므로 값을 옮겨 심는 건 전부 C의 일입니다.

움길 값	출처	목적지	빠뜨리면
<code>run_id</code>	C가 발급	①. <code>reset</code> · ④ 전체	<code>truth.jsonl</code> 조인 불가 (V3)
<code>recent_error</code>	⑤. <code>get_reliability_table</code>	②. <code>propose_allocation</code>	LSTM 신뢰도 계산 불가 (V4)
<code>in_distribution</code>	②. <code>propose_allocation</code>	④. <code>record_decision</code>	논문 주장 검증 불가 (V7)
<code>demand_class</code>	②. <code>classify_demand</code>	④. <code>record_decision</code>	분류기/판단 오류 구분 불가 (V8)
<code>capacity_gain</code>	③. <code>procure</code>	①. <code>add_capacity</code>	조달이 환경에 무반영 (정정 K)
<code>observation.step</code>	①. <code>get_observation</code>	③. <code>procure</code> (<code>current_step</code>)	만료 시점 계산 불가 (W2)
<code>recent_errors</code> (전체)	⑤. <code>get_reliability_table</code>	②. <code>compare_policies</code>	전 정책이 같은 오차 공유 (W6)
<code>vendor_id</code>	⑤. <code>report_outcome</code>	③. <code>update_rating</code>	벤더 레이팅 미갱신

`run_id` 는 C가 만듭니다. 형식은 `{arm}-{scenario}-s{seed}` — 예: `proposed-emergency-s0`. ①과 ④에 같은 값을 넘겨야 `decision_id ( {run_id}-{step:04d} )`가 `truth.jsonl` 과 `step` 으로 조인됩니다.

W1 — 조달은 `record_decision` 전, `step()` 전에 합니다.

```

2b. [조달 분기]  obs.demand_pressure ≥ 1.0 일 때만
    ③.score_offerings → ③.procure(..., current_step=obs.step)
    ①.add_capacity(amount=capacity_gain, expires_at_step)
3. ④.record_decision(..., slice_id, vendor_id)
4. ①.apply_allocation
5. ①.step()

```

`add_capacity` 가 `step()` 뒤로 가면 조달한 바로 그 스텝이 효과를 못 봅니다. 에이전트 입장에서 "샀는데 아무 일도 안 일어났다" 가 되어 조달 학습이 왜곡됩니다.

조달이 기록(3)보다 앞서는 것은 규칙 3("기록이 실행보다 먼저")의 유일한 예외입니다 — `record_decision` 이 `slice_id · vendor_id` 를 인자로 받기 때문입니다. 조달에 실패해도 `slice_id=null` 과 `rationale` 로 기록하세요. 시도했다는 사실 자체가 판단의 증거입니다.

에스컬레이션한 스텝도 루프를 끝까지 돕니다 (V2). `record_escalation` 이 `decision_id` 와 `fallback_allocation` 을 함께 주므로, 그걸로 `apply_allocation` → `step()` → `report_outcome` 을 평소대로 호출하세요. 여기서 멈추면 그 스텝이 통계에서 사라집니다.

신뢰도 종합은 에이전트만 합니다 — 어느 서버에도 이 코드가 없습니다.

```

combined = sqrt(intrinsic × effective)
escalate if combined < TAU (0.45)

```

C-2. 비교군 스위치

`--arm` 하나로 4개 비교군을 만듭니다. **A의 하네스가 이걸 호출합니다.**

<code>--arm</code>	상황 인지	정책 선택	에스컬레이션
<code>baseline</code>	<code>truth.jsonl</code> 에서 주입	고정 <code>rule_based</code>	없음
<code>arm1</code>	에이전트	고정 <code>rule_based</code>	없음
<code>arm2</code>	에이전트	에이전트	없음
<code>proposed</code>	에이전트	에이전트	신뢰도 기반

`baseline` 만 `truth.jsonl` 을 읽습니다. 기존 시스템(CLI `--emergency`)의 재현이므로 정당하지만, **코드 경로를 물리적으로 분리하세요.** 같은 함수에 `if arm == "baseline"` 으로 두면 실수로 다른 비교군에 새어 들어갑니다.

C-3. 프롬프트 설계 — `orchestrator/prompts/`

여기에 시간이 가장 많이 듭니다. 서버 개발과 리듬이 다릅니다.

도구 설명과 정확히 같은 문제가 **프롬프트에도 있습니다** (설계서 §9-9). 시스템 프롬프트에 *"URLLC 이용률이 높으면 비상 상황일 수 있다"* 라고 적으면, 에이전트가 판단한 게 아니라 받아쓴 것이 됩니다.

```
prompts/
├── system_minimal.md      # 역할·도구 사용 규약만. 판단 기준 없음
├── system_advisory.md     # 배경 지식 포함
└── step.md                # 스텝별 사용자 메시지 템플릿
```

주 실험은 `minimal`. `PROMPT_HASH` 를 `decisions.json` 의 `config` 에 기록해 사후 추적 가능하게 하세요.

LLM은 `confidence.situation` 도 함께 답해야 합니다. `intrinsic·empirical` 은 둘 다 정책에 대한 확신이라 상황을 잘못 읽었을 가능성이 어디에도 반영되지 않습니다.

```
combined = (situation × intrinsic × empirical)^(1/3)
escalate if combined < 0.45
```

실측상 상황 인지의 상한이 약 72%, `normal` 은 51%입니다 (`TOOLS.md` §10). **오탐이 구조적으로 많다**는 뜻이므로, 이 경로가 없으면 *"무슨 일인지 모르겠다"* 는 이유로는 영영 사람을 부르지 않습니다.

C-4. 누출 검사 — `orchestrator/leak_check.py`

C만 할 수 있는 일입니다. A가 자기 코드를 아무리 봐도 못 찾습니다. 에이전트의 실제 컨텍스트를 덤프해서 금지어를 찾습니다.

```
FORBIDDEN = ["is_emergency", "is_special_event", "is_iot_surge",
             "perception_accuracy", "escalation_precision", "ground_truth"]
```

`baseline` 을 제외한 모든 실행에서 **0건이어야 합니다.** 1건이라도 나오면 그 실행은 폐기입니다.

C-5. 계약 검사 — docs/contract.json + tools/check_contract.py

도구 계약 검증에서 **값 유실 9건**이 나왔습니다 (설계서 §1.5). 도구는 앞으로도 늘어나고(이미 20 → 21), 사람이 매번 역추적하면 또 놓칩니다.

기계가 읽을 계약을 두고 검사기를 돌립니다. 목 서버와 같은 성격 — **계약의 실행 가능한 명세**이고, C는 A·B의 서버를 소비하는 쪽이라 불일치를 가장 먼저 만납니다.

```
{
  "propose_allocation": {
    "server": "policy", "order": 2,
    "requires": {
      "observation": "observe.get_observation",
      "situation": "AGENT",
      "recent_error": "feedback.get_reliability_table"
    },
    "produces": ["policy", "allocation", "confidence", "in_distribution", "status"]
  }
}
```

검사 4개:

#	검사	차단하는 유형
1	모든 필수 입력에 생산자가 있는가 (AGENT 명시 포함)	V3 · V4
2	생산자의 호출 순서가 소비자보다 앞인가	V5 · V6
3	명세의 모든 식별자에 발급 도구가 있는가	V2
4	아무도 소비하지 않는 출력 필드 목록 (경고)	V7 · V8

추가 검사 — AGENT 가 생산자인 값은 네 개뿐이어야 합니다.

```
situation · confidence.combined · 에스컬레이션 판단 · 정책 선택
```

이 넷이 연구의 기여 지점 전체입니다. 목록이 늘어나면 에이전트가 해야 할 판단이 서버로 새고 있다는 뜻이므로, 넷을 넘으면 경고하도록 하세요. 줄어들어도 경고입니다 — 기여가 사라진 겁니다.

4.2 완료 판정

#	판정
C-0	목 5개로 루프 1회전 완주
C-1	진짜 서버 5개로 60스텝 무인 완주
C-2	비교군 4개가 동일 시드에서 동일 스텝 수 를 소화
C-3	<code>minimal</code> / <code>advisory</code> 두 벌 실행 가능
C-4	<code>proposed</code> 실행의 컨텍스트 덤프에서 금지어 0건
C-5	<code>check_contract.py</code> 가 검사 4개 통과. AGENT 생산 값이 정확히 4개
C-1	120스텝 실행에서 <code>report_outcome</code> 누락 0건 (파이썬이 보장하므로 자동)
C-1	스텝당 프롬프트 토큰이 스텝 수에 무관하게 일정
C-1	에스컬레이션한 스텝도 <code>outcome</code> 이 붙는다 (V2)

C-2의 "동일 스텝 수"가 중요합니다. 에스컬레이션 후에도 에이전트는 `rule_based + situation="normal"` 로 계속 진행합니다. 사람을 기다리지 않습니다. 그래야 모든 비교군이 같은 스텝을 소화해 SLA 위반 비교가 공정해집니다.

4.3 C의 함정

- **도구 호출 루프가 발산할 수 있습니다.** 스텝당 호출 횟수 상한(예: 8회)을 두고, 초과하면 강제 에스컬레이션 처리하세요.
- **비용:** 스텝당 3~5회 × 120스텝 × 60회 ≈ **2~3만 호출**. 모델 선정 전에 토큰 예산을 산정하세요 (설계서 \$9-5, 현재 미해소).
- **LLM은 비결정적입니다.** 환경은 시드로 고정되지만 에이전트는 아닙니다. 그래서 시드 3개 × 반복이 필요합니다. "동일 시드 → 동일 결과"를 요구할 수 있는 건 **A의 ①**까지입니다.